

Computer Architecture Project
Pipelined SIMD Multimedia Unit

Final Project Report

By:

Travis Coffino

ID: 115445347

Pranay Srivastava

ID: 115216824

Department of Electrical and Computer Engineering
Stony Brook University

Date: November 30, 2025

Contents

1 Introduction **4**

2 Background and Theory **6**
2.1 SIMD and Subword Parallelism 6
2.2 Pipelining Fundamentals 6
2.3 Pipeline Hazards and Data Forwarding 7

3 Instruction Set Architecture **8**
3.1 Instruction Formats 8
3.2 Operation Summary 8

4 System Architecture **12**
4.1 Top-Level Datapath 12
4.2 Pipeline Overview 12
4.3 Instruction Fetch (IF) Stage 13
4.4 Instruction Decode (ID) Stage 13
4.5 Execute (EX) Stage 14
4.5.1 R3 SIMD ALU 14
4.5.2 R4 Multiply–Accumulate Unit 15
4.5.3 Forwarding Unit 15
4.6 Writeback (WB) Stage 15
4.7 Control Unit 16

5 VHDL Implementation **17**
5.1 Instruction Buffer (IF Stage) 17
5.2 IF/ID Pipeline Register 17
5.3 Register File (ID Stage) 18
5.4 ID/EX Pipeline Register 19
5.5 Data Forwarding Unit 19
5.6 SIMD ALU (EX Stage) 19
5.7 Writeback Stage (`write_back.vhd`) 20
5.8 Top-Level Pipeline Integration 21

6 Python Assembler **22**
6.1 Purpose and Role in the Toolflow . . . 22
6.2 Instruction Encoding in Python . . . 22
6.3 Parsing Helpers and Register Syntax . . . 24
6.4 Line-Level Assembly and Instruction Tables . . . 24
6.5 Main Driver and Output File Format . . . 25
6.6 Example Assembly and Generated Machine Code . . . 26
6.7 Toolflow Usage and Simulation Procedure . . . 27
6.7.1 Step 1: Write the Assembly Program . . . 27
6.7.2 Step 2: Run the Python Assembler . . . 27
6.7.3 Step 3: Move `instructions.mem` into the Active-HDL Project . . . 28
6.7.4 Step 4: Configure and Run the Simulation . . . 28

7 Testbenches and Waveform Verification **29**
7.1 ALU Self-Checking Testbench . . . 29
7.1.1 Datapath Configuration and DUT Interface . . . 29
7.1.2 Instruction Builder Helper Functions . . . 30
7.1.3 Self-Checking Structure . . . 31
7.1.4 LI Test Sequence . . . 31
7.1.5 R4 and R3 Test Blocks . . . 32
7.2 ALU Waveform Results . . . 32
7.2.1 Load Immediate (LI) . . . 32
7.2.2 R4 INT Mode Waveforms . . . 33
7.2.3 R4 LONG Mode Waveforms . . . 35
7.2.4 R3 SIMD Operation Waveforms . . . 37
7.3 Register File Testbench . . . 43
7.3.1 Stimulus Pattern . . . 44
7.3.2 Waveform Analysis . . . 44
7.3.3 Correctness Summary . . . 45
7.4 Instruction Buffer Testbench . . . 45
7.4.1 Testbench Structure . . . 46
7.4.2 Waveform and Console Output . . . 46
7.4.3 Correctness Summary . . . 47
7.5 Data Forwarding Unit Testbench . . . 48
7.5.1 Testbench Structure . . . 48
7.5.2 Waveform Analysis . . . 49

7.5.3 Correctness Summary 50
7.6 Top-Level Pipeline Testbench 51
7.6.1 Program Under Test 51
7.6.2 Waveform 1: Load-Immediate Sequence 52
7.6.3 Waveform 2: OR and Lane-Update into r15 52
7.6.4 Top-Level Correctness Summary 53

8 Results and Discussion **55**
8.1 Functional Verification Summary 55
8.2 Challenges and Observations 55
8.3 Overall System Behavior 56

9 Conclusion **58**

Chapter 1
Introduction

The purpose of this project is to design, implement, and verify a four-stage pipelined SIMD multimedia processing unit using VHDL/Verilog and modern design CAD tools. The processor is modeled after the subword-parallel architectures found in multimedia extensions such as Intel MMX/SSE and the Sony Cell SPE, which exploit data-level parallelism to accelerate image, audio, and video workloads. By developing a reduced but representative set of multimedia operations, this project reinforces the concepts of subword parallelism, pipeline design, data forwarding, and behavioral/structural HDL modeling.

Modern multimedia applications demand hardware capable of processing large volumes of data efficiently, and SIMD architectures provide a practical solution by exploiting data-level parallelism. Designing a simplified version of such a processor offered an opportunity to bridge theoretical concepts from class with real hardware implementation. This project allowed us to explore how architectural features like pipelining, forwarding, and packed computation work together to improve performance, while gaining hands-on experience with VHDL and digital design tools used in industry.

The unit is organized as a standard IF–ID–EX–WB pipeline, with each stage implemented as an independent module separated by interstage registers. Our implementation was done solely in VHDL (Very High-Speed Integrated Circuit Hardware Description Language). The unit supports up to three 128-bit source operands and one 128-bit destination, with arithmetic and logical SIMD operations performed in parallel across subword lanes. The design incorporates a 32×128-bit register file with three read ports and one write port. Instructions are fed into a 64-entry instruction buffer for program storage. A functional data forwarding unit ensures hazard-free execution without stalling, even when dependent instructions appear in back-to-back cycles.

Part 1 of the project focused on the multimedia ALU, which occupies the Execute (EX) stage of the pipeline. This stage is responsible for performing all packed arithmetic and logical operations, and it must do so after taking forwarded values into account. This portion of the project emphasized not only functional correctness but also edge-case behavior such as signed versus unsigned arithmetic, overflow and saturation conditions, and lane-wise shifts and rotations. We implemented the ALU using a behavioral VHDL model and verified it through extensive simulation using Aldec's Active-HDL Student Edition, combining source code, a self-checking testbench, and waveform analysis.

The ALU supports three major instruction classes, distinguished by the high-level instruction format bits:

- **0 – Load Immediate (LI):** Writes a 16-bit immediate value into a selected 16-bit lane of a 128-bit register.
- **10 – R4 Format (Multiply–Accumulate Style):** Performs saturated multiply–add/subtract operations using packed subwords.
- **11 – R3 Format (SIMD Operations):** Includes operations such as halfword/word arithmetic, logic, shifting, rotation, broadcasting, and saturation handling.

Each instruction class has a distinct encoding format, operand handling, and datapath flow.

For the complete system, a modular approach was adopted: each stage was designed, simulated, and verified independently prior to integration into the top-level pipelined processor. To convert the MIPS-style code into a valid 25-bit binary instruction stream, a custom assembler, implemented in Python, was developed. The converted values are written to a `.txt` file that is read by the pipelined processor and stored in the instruction buffer. The design is then exercised cycle-by-cycle, and the final register file contents are compared against the expected results.

This report presents the design, operational diagrams, VHDL source code, simulation waveforms, description of forwarding behavior, testbench outputs, and verification of correct program execution. The final design demonstrates a complete structural RTL implementation of a pipelined SIMD multimedia processor.

Chapter 2
Background and Theory

2.1 SIMD and Subword Parallelism

Single Instruction, Multiple Data (SIMD) architectures perform the same operation on multiple data elements simultaneously. Instead of processing values one at a time, SIMD units pack several subwords (for example, 16-, 32-, or 64-bit elements) into a wider register and apply arithmetic or logical operations in parallel across all lanes.

This technique is widely used in multimedia applications such as graphics, video encoding, signal processing, and image manipulation. Commercial examples include Intel MMX/SSE and the Sony Cell SPE, both of which execute packed operations to accelerate throughput. Our design emulates these concepts using 128-bit registers and parallel lane-based computation.

2.2 Pipelining Fundamentals

Pipelining increases processor throughput by dividing instruction execution into multiple stages and allowing several instructions to be processed concurrently. The four stages used in this project are:

- **IF (Instruction Fetch):** Read the next instruction from the instruction buffer.
- **ID (Instruction Decode):** Decode opcode and operands, read register file sources.
- **EX (Execute):** Perform ALU operations in parallel lanes, using forwarded values if needed.
- **WB (Write Back):** Commit results to the register file.

With proper design, four different instructions can occupy the four pipeline stages simultaneously, achieving one completed instruction per cycle after the pipeline fills.

2.3 Pipeline Hazards and Data Forwarding

Pipeline hazards occur when multiple instructions interact in ways that break the simple pipeline flow. This project focuses primarily on **data hazards**, which arise when an instruction depends on the result of a previous instruction that has not yet been written back.

To eliminate stalls, our design includes a fully functional **data forwarding unit**. Forwarding ensures that:

- The most recent value of a register is always used.
- EX-stage operations can receive results from earlier EX or WB stages.
- Back-to-back dependent instructions execute without stalling.

NOP and certain immediate-based operations do not require forwarding because they lack register sources.

Chapter 3
Instruction Set Architecture

3.1 Instruction Formats

Each instruction is encoded as a 25-bit word `instr[24:0]`. The most significant bits select the instruction format and operation class, while the remaining bits specify register indices, lane selectors, and immediates.

To make the field layout clear, we show the three formats using the diagrams in Figures 3.1–3.3.

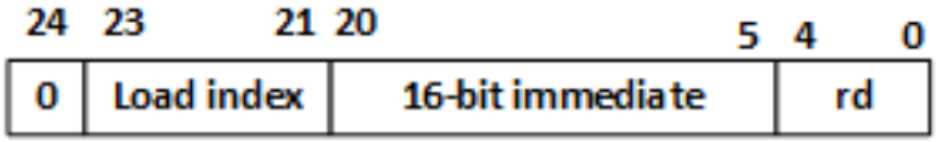


Figure 3.1: LI instruction format (25 bits). Bit 24 selects the LI class; the load index chooses the 16-bit lane of the 128-bit register, and `rd` is the destination register index.

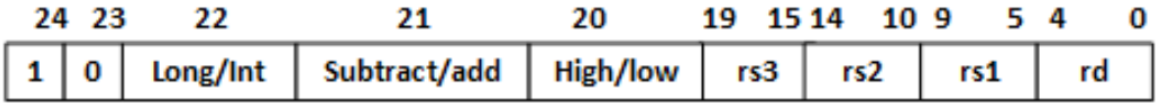


Figure 3.2: R4 instruction format (25 bits) for multiply–accumulate operations. Control bits select long/int (bit-length), add/subtract behavior, and high/low half selection; `rs1`, `rs2`, `rs3`, and `rd` are 5-bit register indices.

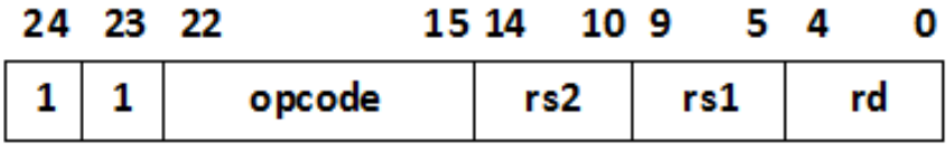


Figure 3.3: R3 instruction format (25 bits) for general SIMD operations. The `opcode` field selects the specific packed arithmetic, logical, shift, rotate, or saturation operation.

3.2 Operation Summary

The three formats support a range of multimedia operations, including load immediate, packed arithmetic, logical operations, shifts/rotates, broadcast, and multiply–accumulate

with saturation. A compact summary of the key fields is given in Table 3.1, while the detailed R4 and R3 opcode semantics are shown in Figures 3.4 and 3.5.

Format	**Opcode / Fields**	**Mnemonic / Use**
LI	idx = instr[23:21], imm = instr[20:5], rd = instr[4:0]	`LI rd, imm, idx` (load 16-bit immediate into lane `idx` of `rd`)
R4	LONG = instr[22], SUB = instr[21], HIGH = instr[20]; rs1, rs2, rs3, rd are 5-bit register indices	MAC / MSUB (INT or LONG): saturated multiply–add/subtract using rs3 and rs2, accumulated into rs1 and written to rd
R3	opcode = instr[18:15], rs1 = instr[9:5], rs2 = instr[14:10], rd = instr[4:0]	SIMD ALU ops (OR, AND, add with saturation, count ones, rotate, min/max, etc.)

Table 3.1: Summary of the three instruction formats in the SIMD multimedia ALU.

LI/SA/HL [22:20]	Description of Instruction Code
000	**Signed Integer Multiply-Add Low with Saturation**: Multiply low 16-bit-fields of each 32-bit field of registers **rs3** and **rs2**, then add 32-bit products to 32-bit fields of register **rs1**, and save result in register **rd**
001	**Signed Integer Multiply-Add High with Saturation**: Multiply high 16-bit-fields of each 32-bit field of registers **rs3** and **rs2**, then add 32-bit products to 32-bit fields of register **rs1**, and save result in register **rd**
010	**Signed Integer Multiply-Subtract Low with Saturation**: Multiply low 16-bit-fields of each 32-bit field of registers **rs3** and **rs2**, then subtract 32-bit products from 32-bit fields of register **rs1**, and save result in register **rd**
011	**Signed Integer Multiply-Subtract High with Saturation**: Multiply high 16-bit- fields of each 32-bit field of registers **rs3** and **rs2**, then subtract 32-bit products from 32-bit fields of register **rs1**, and save result in register **rd**
100	**Signed Long Integer Multiply-Add Low with Saturation**: Multiply low 32-bit- fields of each 64-bit field of registers **rs3** and **rs2**, then add 64-bit products to 64-bit fields of register **rs1**, and save result in register **rd**
101	**Signed Long Integer Multiply-Add High with Saturation**: Multiply high 32-bit- fields of each 64-bit field of registers **rs3** and **rs2**, then add 64-bit products to 64-bit fields of register **rs1**, and save result in register **rd**
110	**Signed Long Integer Multiply-Subtract Low with Saturation**: Multiply low 32- bit-fields of each 64-bit field of registers **rs3** and **rs2**, then subtract 64-bit products from 64-bit fields of register **rs1**, and save result in register **rd**
111	**Signed Long Integer Multiply-Subtract High with Saturation**: Multiply high 32- bit-fields of each 64-bit field of registers **rs3** and **rs2**, then subtract 64-bit products from 64-bit fields of register **rs1**, and save result in register **rd**

Figure 3.4: R4 multiply–accumulate instruction encodings and descriptions (from project handout).

Opcode [22:15]	Description of Instruction Opcode
xxxx0000	**NOP:** *no operation*. Make sure that a NOP instruction does not write anything to the register file!
xxxx0001	**SHRHI**: *shift right halfword immediate*: packed 16-bit halfword shift right logical of the contents of register ***rs1*** by the value of the 4 least significatn bits of instruction field ***rs2***. Each of the results is placed into the corresponding 16-bit slot in register ***rd***. Bits shifted out for each halfword are dropped, and bits shifted in to each halfword should be zeros. (Comments: 8 separate 16-bit values in each 128-bit register)
xxxx0010	**AU**: ***add word unsigned***: packed 32-bit unsigned addition of the contents of registers ***rs1*** and ***rs2*** (Comments: 4 separate 32-bit values in each 128-bit register)
xxxx0011	**CNT1H**: ***count 1s in halfword***: count 1s in each packed 16-bit halfword of the contents of register ***rs1.*** The results are placed into corresponding slots in register ***rd***. (Comments: 8 separate 16-bit values in each 128-bit register)
xxxx0100	**AHS**: ***add halfword saturated***: packed 16-bit halfword signed addition **with saturation** of the contents of registers ***rs1*** and ***rs2*** . (Comments: 8 separate 16-bit values in each 128-bit register)
xxxx0101	**OR:** ***bitwise logical or*** of the contents of registers ***rs1*** and ***rs2***
xxxx0110	**BCW**: ***broadcast word***: broadcast the leftmost 32-bit word of register ***rs1*** to each of the four 32-bit words of register ***rd***
xxxx0111	**MAXWS**: ***max signed word***: for each of the four 32-bit word slots, place the maximum signed value between ***rs1*** and ***rs2*** in register ***rd***. (Comments: 4 separate 32-bit values in each128-bit register)
xxxx1000	**MINWS**: ***min signed word***: for each of the four 32-bit word slots, place the minimum signed value between ***rs1*** and ***rs2*** in register ***rd*** . (Comments: 4 separate 32-bit values in each 128-bit register)
xxxx1001	**MLHU**: ***multiply low unsigned***: the 16 rightmost bits of each of the four 32-bit slots in register ***rs1*** are multiplied by the 16 rightmost bits of the corresponding 32-bit slots in register ***rs2***, treating both operands as unsigned. The four 32-bit products are placed into the corresponding slots of register ***rd*** . (Comments: 4 separate 32-bit values in each 128-bit register)
xxxx1010	**MLHCU**: ***multiply low by constant unsigned***: the 16 rightmost bits of each of the four 32-bit slots in register ***rs1*** are multiplied by a 5-bit value in the ***rs2*** field of the instruction, treating both operands as unsigned. The four 32-bit products are placed into the corresponding slots of register ***rd*** . (Comments: 4 separate 32-bit values in each 128-bit register)
xxxx1011	**AND**: ***bitwise logical and*** of the contents of registers ***rs1*** and ***rs2***
xxxx1100	**CLZW**: ***count leading zeroes in words*:** for each of the four 32-bit word slots in register **rs1,** count the number of zero bits to the left of the first "1". If the word slot in register **rs1** is zero, the result is 32. The four results are placed into the corresponding 32-bit word slots in register **rd**. (Comments: 4 separate 32-bit values in each 128-bit register)
xxxx1101	**ROTW**: ***rotate bits in word*** : the contents of each 32-bit field in register ***rs1*** are rotated to the right according to the value of the 5 least significant bits of the corresponding 32-bit field in register ***rs2***. The results are placed in register ***rd***. Bits rotated out of the right end of each word are rotated in on the left end of the same 32-bit word field. (Comments: 4 separate 32-bit word values in each 128-bit register)
xxxx1110	**SFWU**: ***subtract from word unsigned***: packed 32-bit word unsigned subtract of the contentsof ***rs1*** from ***rs2*** (rd = rs2 - rs1). (Comments: 4 separate 32-bit values in each 128-bit register)
xxxx1111	**SFHS**: ***subtract from halfword saturated***: packed 16-bit halfword signed subtraction **with saturation** of the contents of ***rs1*** from ***rs2*** (rd = rs2 - rs1). (Comments: 8 separate 16-bit values in each 128-bit register)

Figure 3.5: R3 SIMD ALU opcode encodings and descriptions (from project handout).

Chapter 4
System Architecture

4.1 Top-Level Datapath

Figure 4.1 shows the top-level datapath of the pipelined SIMD multimedia unit, including the instruction buffer, pipeline registers, register file, data forwarding unit, and ALU.

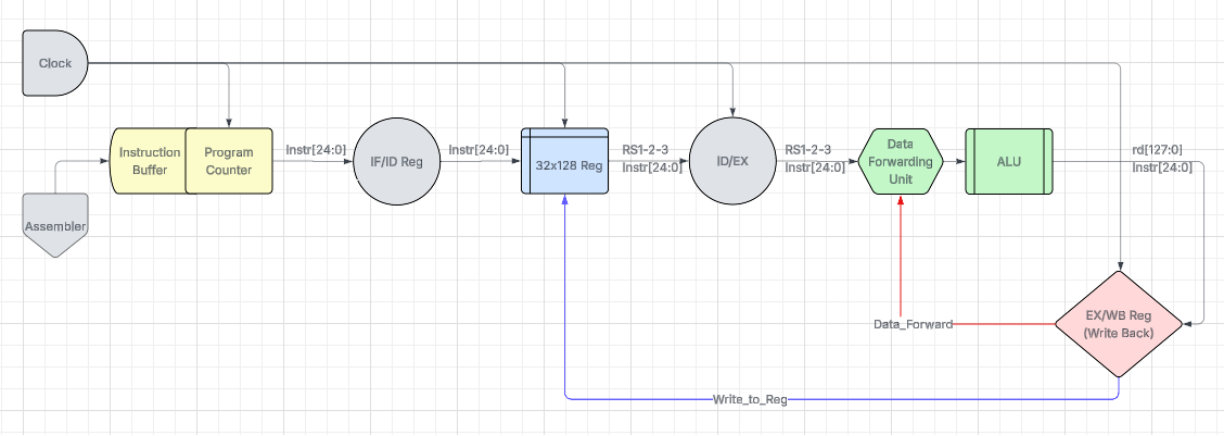


Figure 4.1: Top-level architecture of the four-stage pipelined SIMD multimedia unit.

4.2 Pipeline Overview

The pipelined SIMD multimedia processor is organized into four stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), and Writeback (WB). Each stage is isolated by pipeline registers that store both data operands and control signals. After the pipeline is filled, one instruction is completed per cycle under ideal conditions.

The datapath supports three instruction formats (LI, R3, R4), each with a distinct operand structure and execution flow. The pipeline must therefore accommodate multiple operand widths (16-, 32-, and 64-bit packed elements), three-operand R4 operations, lane-select immediates for LI, and forwarding of recent results to avoid pipeline stalls. This section provides a detailed description of each stage, the registers between stages, and the datapath components that interact to form the complete SIMD pipeline.

4.3 Instruction Fetch (IF) Stage

The Instruction Fetch stage supplies one 25-bit instruction per cycle to the pipeline. Instructions are stored in a 64-entry instruction buffer implemented as a simple synchronous ROM with a single read port. The program counter (PC) indexes into this buffer and increments sequentially; branching is not supported in this design, so the IF stage has no control hazards.

During IF the following values are produced:

- **instr_in**: the 25-bit instruction fetched from the buffer.
- **pc_next**: PC + 1, for sequential execution.

These values are latched into the **IF/ID** pipeline register, which stores:

- The fetched instruction.
- The next PC index.

This stage is purely structural and contains no combinational datapath beyond address generation.

4.4 Instruction Decode (ID) Stage

The ID stage decodes the 25-bit instruction and reads operand values from the 32×128-bit register file. The decode logic extracts fields differently depending on the instruction class:

- LI: lane index (bits 23–21), 16-bit immediate (20–5), rd (4–0).
- R3: opcode (18–15), rs2 (14–10), rs1 (9–5), rd (4–0).
- R4: mode bits (22–20), rs3 (19–15), rs2 (14–10), rs1 (9–5), rd (4–0).

The register file provides three read ports to support the operand demands of R4 instructions. For LI, only a 16-bit immediate is used; for R3 and R4 instructions, one to three source operands are read.

A dedicated **lane-select mux** prepares immediate data for LI. The 128-bit destination value is assembled by writing the 16-bit immediate into the appropriate lane and passing the remaining bits unchanged.

The **ID/EX** pipeline register stores:

- rs1_val, rs2_val, rs3_val (128-bit operands).
- Control bits for the ALU or MAC unit.
- Immediate values for LI or shift operations.
- Destination register index (rd).

4.5 Execute (EX) Stage

The EX stage performs all arithmetic and logical operations. It contains two major subunits:

1. The **R3 SIMD ALU**, which performs packed arithmetic, logical, and shift/rotate operations.
2. The **R4 MAC unit**, which performs long or integer saturated multiply–accumulate operations on halfword or word subfields.

A control multiplexer selects between the R3 and R4 datapaths based on the instruction's top two bits. LI instructions bypass the ALU entirely and are handled by a dedicated immediate-lane insertion path.

4.5.1 R3 SIMD ALU

The R3 ALU operates on packed subwords within each 128-bit operand. The ALU supports:

- Packed addition/subtraction with saturation (AHS).
- Bitwise operations (AND, OR).
- Rotations and shifts (ROTW, SHRHI).
- Bit counting (CNT1H).
- Saturating multiply/low-high halfword extraction (MLHU, MLHCU).

Each operation is computed independently in parallel lanes. Control logic configures the 4-bit opcode extracted from the R3 format to activate the correct lane-slice operator.

Forwarding muxes feed rs1 and rs2 with either:

- Values fetched in ID, or
- Forwarded values from the EX/WB stage.

4.5.2 R4 Multiply–Accumulate Unit

R4 instructions perform saturated multiply–add or multiply–subtract across packed 16-bit or 32-bit lanes:

- **LONG bit:** selects 32-bit or 16-bit multiplies.
- **SUB bit:** chooses add vs. subtract.
- **HIGH bit:** selects the high or low half of the product.

The unit multiplies the appropriate lane slices of `rs3` and `rs2`, extracts the selected half of the product, and then adds or subtracts the result from the accumulator value `rs1`. The result is saturated to the lane width before being forwarded to the EX/WB register.

4.5.3 Forwarding Unit

To avoid RAW hazards, the forwarding unit compares the destination register of an instruction in WB against the source registers of the instruction currently in EX. If a match occurs and write-enable is asserted, the forwarded value is selected through:

- forwardA (feeds rs1 input to ALU/MAC),
- forwardB (feeds rs2 input),
- forwardC (feeds rs3 input for R4 operations).

Forwarding takes priority over ID-register-file outputs, ensuring correct execution of back-to-back dependent instructions without pipeline stalls.

4.6 Writeback (WB) Stage

The final stage writes results back to the register file. The **EX/WB** pipeline register supplies:

- The computed 128-bit value.
- Destination register index (rd).
- Lane index for LI operations.
- Write enable signal.

R3 and R4 instructions write a full 128-bit value. LI replaces only a single 16-bit lane, preserving all other bits of the destination register.

Writeback completes the instruction's lifecycle and makes its result available for subsequent instructions, including forwarding paths for those entering EX.

4.7 Control Unit

The control unit decodes the instruction class based on bits 24–23:

- **0:** LI format.
- **10:** R4 multiply–accumulate format.
- **11:** R3 SIMD ALU format.

For R4 instructions, additional mode bits (`LONG`, `SUB`, `HIGH`) configure the MAC unit. For R3 instructions, the 4-bit opcode (bits 18–15) selects the specific SIMD operation.

The control unit generates all necessary control signals, including:

- ALU/MAC select.
- Opcode selection.
- Lane index for LI.
- Register file write enable.
- Forwarding select signals.

Together, these control paths coordinate the behavior of the datapath modules and ensure correct execution of all instruction formats.

Chapter 5
VHDL Implementation

This chapter presents the hardware implementation of the four-stage pipelined SIMD multimedia processor. Each pipeline stage is implemented as a standalone VHDL module, with clearly defined interfaces and structural composition at the top level. The design follows a clean IF–ID–EX–WB architectural pattern, with three pipeline registers, a 32×128-bit register file, a fully combinational SIMD ALU, and a unified forwarding network.

The following sections summarize the purpose, behavior, and selected excerpts of the key VHDL modules used to construct the processor.

5.1 Instruction Buffer (IF Stage)

The `instr_buffer.vhd` module serves as the program memory for the processor. It stores 64 instructions, each encoded as a 25-bit word. At simulation time, the memory is initialized automatically by reading `instructions.mem` line-by-line using VHDL `textio`. Each line in the file must contain exactly 25 binary characters corresponding to one instruction.

The instruction buffer outputs:

- the current program counter (`pc_out`), and
- the instruction at that PC (`instr_out`).

A simplified excerpt is shown below:

```
instr_out <= mem(to_integer(pc));
pc_out    <= pc;
pc <= pc + 1;
```

The PC wraps around at 63, implementing a simple circular program memory.

5.2 IF/ID Pipeline Register

The `reg_if_id.vhd` module provides the first pipeline boundary, latching the instruction and program counter as they move from instruction fetch into decode. This module is purely synchronous and contains no combinational logic other than basic forwarding of signal values.

```
if rising_edge(clk) then
    instr_out <= instr_in;
    pc_out    <= pc_in;
end if;
```

This ensures a stable, cycle-by-cycle flow of instruction information into the decode stage.

5.3 Register File (ID Stage)

The `registers.vhd` module implements the architectural register file:

- 32 registers,
- each 128 bits wide,
- 3 simultaneous read ports,
- 1 synchronous write port.

Read operations are combinational, providing zero-cycle access to operands during the ID stage. Writeback occurs on the rising edge of the writeback clock.

Operand selection depends on the instruction format:

- **LI**: only `rs1` is read (the destination's previous value).
- **R4**: `rs1`, `rs2`, and `rs3` are read.
- **R3**: `rs1` and `rs2` are read.

Example excerpt:

```
if instructioni(24) = '0' then          -- LI
    rs1 <= fetch_data(instructioni(4 downto 0), mem);

elsif instructioni(24 downto 23) = "10" then  -- R4
    rs1 <= fetch(...)
    rs2 <= fetch(...)
    rs3 <= fetch(...)
```

This register file provides fully parallel operand fetch for all instruction types.

5.4 ID/EX Pipeline Register

The `reg_id_ex.vhd` module forms the second pipeline boundary, latching the decoded operands and instruction into the execution stage. This stage has no control logic and simply carries data forward into EX.

```
rs1 <= rs1in;
rs2 <= rs2in;
rs3 <= rs3in;
instruction <= instructioni;
```

This guarantees deterministic behavior for the ALU and forwarding network.

5.5 Data Forwarding Unit

To avoid pipeline stalls, the processor includes a unified forwarding unit implemented in `data_forwarding_unit.vhd`. This block compares the source register fields of the instruction in EX against:

- the destination register of the instruction in EX (EX-stage forwarding)
- the destination register of the instruction in WB (WB-stage forwarding)

If a match is detected and the write-enable flag is asserted, the forwarded data overrides the original operand value.

Example logic for R3 forwarding:

```
if ex_we='1' and instr_id(9 downto 5)=ex_addr then
    rs1 <= ex_data;
elsif wb_we='1' and instr_id(9 downto 5)=wb_addr then
    rs1 <= wb_data;
end if;
```

The result is a completely stall-free pipeline for all valid instruction sequences containing data dependencies.

5.6 SIMD ALU (EX Stage)

The ALU, implemented in `alu.vhd`, is the computational core of the processor. It is fully combinational and supports all three instruction formats:

- **LI**: insert a 16-bit immediate into a chosen 16-bit lane.
- **R4**: saturated multiply–accumulate operations over 16-bit or 32-bit subwords, depending on mode bits.
- **R3**: general SIMD operations (logic, shifts, rotates, saturating adds/subtracts, leading-zero count, broadcast, etc.).

Internally, the ALU splits inputs into subword slices:

- 8 lanes of 16 bits for halfword operations,
- 4 lanes of 32 bits for word operations,
- 2 lanes of 64 bits for R4 long/int multiply–accumulate.

Example R3 logic for saturated 16-bit addition:

```
r3_r16 := clamp16(resize(r3_a16,17) + resize(r3_b16,17));
```

Example R4 MAC logic (INT mode):

```
prod32 := signed(a16) * signed(b16);
acc64  := resize(old32,64) + resize(prod32,64);
dst_64(field) := sat32(acc64);
```

The ALU outputs a single 128-bit result `rd`, which is clocked into the writeback pipeline register.

5.7 Writeback Stage (`write_back.vhd`)

The final stage of the pipeline is implemented in the `write_back.vhd` module. It does not perform arithmetic; instead, it packages the metadata required for register file updates and WB forwarding.

On each rising clock edge, it latches:

- **df_addr**: the destination register index (`rd` field)
- **df_data**: the ALU's 128-bit result
- **wr_en**: whether the instruction writes back

The `will_write` function implements the writeback policy:

- LI and R4 always write,
- R3 writes unless opcode = `NOP`,
- illegal formats do not write.

These signals feed both the register file's write port and the forwarding unit, enabling future instructions to observe the correct data without delays.

5.8 Top-Level Pipeline Integration

The complete structural design is assembled in `pipelined_simd_multimedia_unit.vhd`, which instantiates:

- the instruction buffer,
- all three pipeline registers,
- the register file,
- the forwarding unit,
- the ALU, and
- the writeback block.

Four clock signals (`clk_if`, `clk_id`, `clk_ex`, `clk_wb`) allow each stage to be independently controlled for simulation visibility.

Data flows cleanly from IF to WB on every cycle, and forwarding resolves all RAW hazards, yielding a fully functional and stall-free SIMD pipeline.

Chapter 6
Python Assembler

6.1 Purpose and Role in the Toolflow

To exercise the pipelined SIMD multimedia unit with realistic programs, we developed a custom Python assembler that translates a human-readable assembly language into the 25-bit binary instruction format required by the VHDL instruction buffer. The assembler:

- parses a `.asm` source file containing LI, R3, and R4 mnemonics,
- encodes each instruction into a 25-bit integer according to the ISA defined in Chapter 3, and
- writes one 25-bit binary string per line into `instructions.mem`.

The VHDL module `instr_buffer.vhd` consumes this file at simulation startup using `textio`, so the Python assembler forms a crucial bridge between the software-level program and the hardware-level execution of the pipeline.

6.2 Instruction Encoding in Python

The assembler directly mirrors the three instruction formats used in the hardware. Each format is implemented as a separate encoding function:

- `encode_li(rd, lane, imm16)`
- `encode_r4(longInt, subAdd, highLow, rd, rs1, rs2, rs3)`
- `encode_r3(opcode, rd, rs1, rs2_or_imm5)`

For example, the LI encoder sets bit 24 to zero, inserts the 3-bit lane index, the 16-bit immediate, and the 5-bit destination register:

```
def encode_li(rd, lane, imm16):
    instr = 0
    instr |= 0 << 24
    instr |= (lane & 0x7)    << 21
```

```
    instr |= (imm16 & 0xFFFF) << 5
    instr |= (rd & 0x1F)
    return instr
```

The R4 encoder sets the top bits to `10` to select the MAC family, then packs the three mode bits and four register indices:

```
def encode_r4(longInt, subAdd, highLow, rd, rs1, rs2, rs3):
    instr = 0
    instr |= 0b10 << 23           # bits 24..23 = "10"
    form = ((longInt & 1) << 2) | \
           ((subAdd  & 1) << 1) | \
           ( highLow & 1)
    instr |= (form & 0x7)  << 20  # bits 22..20
    instr |= (rs3  & 0x1F) << 15  # bits 19..15
    instr |= (rs2  & 0x1F) << 10  # bits 14..10
    instr |= (rs1  & 0x1F) << 5   # bits 9..5
    instr |= (rd   & 0x1F)        # bits 4..0
    return instr
```

In this convention, the assembly syntax for R4 instructions is `MODE rd, rs1, rs2, rs3`, but for encoding purposes the rightmost operand (`rs3`) is placed in the most significant source field (`instr[19:15]`), `rs2` in the next field, and `rs1` in `instr[9:5]`. The comments in the assembler and ALU carefully document this right-to-left mapping so that the Python tool and the VHDL datapath agree.

Similarly, the R3 encoder sets the top bits to `11` and packs the 4-bit opcode and the two source fields:

```
def encode_r3(opcode, rd, rs1, rs2_or_imm5):
    instr = 0
    instr |= 0b11 << 23            # bits 24..23 = "11"
    instr |= (opcode & 0xF) << 15  # bits 18..15
    instr |= (rs2_or_imm5 & 0x1F) << 10
    instr |= (rs1 & 0x1F) << 5
    instr |= (rd  & 0x1F)
    return instr
```

6.3 Parsing Helpers and Register Syntax

To keep the main assembler logic clean, several small helper functions handle token cleanup and integer parsing:

- `remove_comments(line)` strips off anything following `#` or VHDL-style `--`.
- `clean_token(tok)` removes trailing commas and whitespace.
- `parse_int(token)` accepts decimal, hexadecimal (`0x...`), or binary (`0b...`) literals.
- `parse_reg(token)` accepts both plain numbers (e.g., `16`) and register names with an `r` prefix (e.g., `r16` or `R17`).

This allows assembly code to be written in a readable style, such as:

```
LI   r16, 0, 0x0001
OR   r15, r16, r17
INT_LOW_ADD  r20, r16, r17, r18
```

6.4 Line-Level Assembly and Instruction Tables

The core of the assembler is the `assemble_line` function, which takes a raw source line and a line number, strips comments, tokenizes the mnemonic and operands, and dispatches to the appropriate encoder.

For R4 instructions, a dictionary maps mnemonic strings to the three mode bits `(longInt, subAdd, highLow)`. For example:

```
r4_modes = {
    "INT_LOW_ADD"   : (0, 0, 0),
    "INT_HIGH_ADD"  : (0, 0, 1),
    "INT_LOW_SUB"   : (0, 1, 0),
    "INT_HIGH_SUB"  : (0, 1, 1),
    "LONG_LOW_ADD"  : (1, 0, 0),
    "LONG_HIGH_ADD" : (1, 0, 1),
    "LONG_LOW_SUB"  : (1, 1, 0),
    "LONG_HIGH_SUB" : (1, 1, 1),
}
```

Any mnemonic appearing in this table is treated as an R4 MAC instruction with syntax:

```
MODE rd, rs1, rs2, rs3
```

The assembler extracts the registers with `parse_reg` and forwards them to `encode_r4` along with the mode bits.

For R3 operations, a similar dictionary maps mnemonic names to a pair `(opcode, form)`, where `form` describes the operand pattern:

- `"nop"` for a bare `NOP`,
- `"rd_rs1"` for unary operations (e.g., `CNT1H`),
- `"rd_rs1_rs2"` for binary operations,
- `"rd_rs1_imm5"` for instructions such as `SHRHI` and `MLHCU` that use an immediate shift or scale amount.

The `assemble_line` routine checks the form and validates the number of operands, producing clear error messages when the syntax does not match the expected pattern (e.g., wrong operand count or unknown mnemonic). This makes it easier to debug assembly programs before they reach the VHDL simulation.

6.5 Main Driver and Output File Format

The `main()` function implements the overall workflow:

1. Prompt the user for the input assembly filename (e.g., `program.asm`).
2. Open the source file and iterate over lines, calling `assemble_line` for each.
3. For each non-`None` instruction, format the 25-bit integer as a binary string:

```
bits = f"instr:025b"
```

4. Append each 25-bit binary string to a list.
5. Write all binary strings, one per line, into the fixed output file `instructions.mem`.

At the end of execution, the assembler prints the number of instructions assembled and confirms the output filename:

```
Wrote N instructions to instructions.mem
```

This file is then consumed directly by the `instr_buffer.vhd` module at simulation startup, providing a clean, automated path from human-readable assembly to the binary machine code seen by the pipeline.

6.6 Example Assembly and Generated Machine Code

As a simple example, consider the following assembly program:

```
# Load r16 lane0 = 1, r17 lane0 = 2
LI  r16, 0, 0x0001
LI  r17, 0, 0x0002

# OR r15 = r16 OR r17
OR  r15, r16, r17

# Accumulate r20 with r16 * r17
INT_LOW_ADD  r20, r16, r17, r18

# Shift-right each halfword of r10 by 3
SHRHI r10, r10, 3

NOP
```

Running the Python assembler on this file produces a corresponding `instructions.mem` with one 25-bit binary word per line, for example:

```
0000000000000000000010000
0000000000000000000100001
1101010001000101000011111
1000010000100011001010100
1100000010000100000000000
0000000000000000000000000
```

(Exact bit patterns depend on the register indices and opcode encodings specified in Chapter 3, but the assembler guarantees that the mapping between mnemonics and 25-bit words is consistent with the VHDL implementation.)

This demonstrates the complete toolflow: a compact assembly program is written in a readable, mnemonic-based language; the Python assembler converts it into a 25-bit binary instruction stream; and the VHDL instruction buffer loads and feeds those instructions into the pipelined SIMD multimedia unit for cycle-accurate simulation.

6.7 Toolflow Usage and Simulation Procedure

This section summarizes the complete workflow from writing an assembly program to observing the pipelined execution in Active-HDL.

6.7.1 Step 1: Write the Assembly Program

First, the user creates a `.asm` file containing the desired sequence of LI, R3, and R4 instructions. This source file must be placed in the same folder as the Python assembler script (e.g., `assembler_v2.py`). A simple example might be:

```
# Example program
LI   r16, 0, 0x0001
LI   r17, 0, 0x0002
OR   r15, r16, r17
INT_LOW_ADD  r20, r16, r17, r18
SHRHI r10, r10, 3
NOP
```

6.7.2 Step 2: Run the Python Assembler

With the source file in place, the assembler is invoked from a terminal or command prompt. The script prompts for the input `.asm` filename, reads and assembles each line, and then automatically writes the encoded machine code to a file named `instructions.mem` in the same directory.

After successful assembly, the terminal prints a short summary such as:

```
Wrote N instructions to instructions.mem
```

The `instructions.mem` file now contains one 25-bit binary instruction per line and is ready to be used by the VHDL simulation.

6.7.3 Step 3: Move `instructions.mem` into the Active-HDL Project

The generated `instructions.mem` file is then copied into the Active-HDL project's source directory that contains all of the VHDL files for the pipelined SIMD multimedia unit. The file name must remain `instructions.mem`, since the `instr_buffer.vhd` module opens this exact file using `textio` at elaboration time.

Once copied, no further changes are required; different programs can be tested simply by regenerating and replacing `instructions.mem`.

6.7.4 Step 4: Configure and Run the Simulation

In Active-HDL, the top-level unit for full-system verification is `tb_multimedia_pipeline.vhd`, which instantiates the `pipelined_simd_multimedia_unit` and provides clock and reset generation.

To run the complete pipeline simulation, the user:

1. Sets `tb_multimedia_pipeline` as the top-level design.
2. Opens a waveform window and adds the desired internal signals (e.g., PCs, register file ports, ALU result, forwarding signals).
3. Starts the simulation. The testbench runs for a fixed number of cycles and then terminates automatically.

At the end of execution, the waveform viewer shows the cycle-by-cycle behavior of the pipeline, including instruction flow through the IF/ID/EX/WB stages, forwarded values, and final writebacks to the register file.

The following chapter describes the individual unit testbenches used to verify each component in isolation, as well as the integrated pipeline testbench and its resulting waveforms.

Chapter 7
Testbenches and Waveform Verification

This chapter describes the VHDL testbenches used to verify the correctness of each module in the pipelined SIMD multimedia unit and presents the corresponding waveforms. All non-trivial components were validated using *self-checking* testbenches that automatically compare the device-under-test (DUT) output against a golden reference value and raise an assertion failure on any mismatch.

The most complex of these is the ALU testbench, which exhaustively exercises the load-immediate (LI), R4 (multiply–accumulate), and R3 (packed SIMD) instruction classes.

7.1 ALU Self-Checking Testbench

The ALU is verified using a standalone self-checking testbench `alu_sc_tb.vhd`. This testbench instantiates the 128-bit SIMD ALU as the DUT and drives it with a sequence of LI, R4, and R3 instructions, together with carefully chosen 128-bit operand patterns. For each test case, the expected 128-bit result is computed offline and hard-coded into the testbench as a golden reference vector.[1]

7.1.1 Datapath Configuration and DUT Interface

At the top of `alu_sc_tb.vhd`, two constants define the ALU datapath shape:

- `LANE_W = 64`: each lane is 64 bits wide.
- `LANES = 2`: there are two lanes, for a total width of 128 bits.

The DUT is instantiated with the following ports:

- `instruction : std_logic_vector(24 downto 0)` — the 25-bit instruction word.
- `rs1, rs2, rs3 : std_logic_vector(127 downto 0)` — three 128-bit source operands.
- `rd : std_logic_vector(127 downto 0)` — the 128-bit ALU result.

Within the testbench, these ports are declared as signals and directly mapped into the ALU entity `work.alu`.

[1] Because the ALU is purely combinational, no clock is required in this testbench; each input pattern is applied for a short simulation delay (e.g., 1–5 ns) before checking the output.

7.1.2 Instruction Builder Helper Functions

To avoid manually writing bit-level encodings for every test, the testbench provides three helper functions that generate valid 25-bit instructions:

- `mk_li(lane, imm16)`: Constructs an LI instruction with:
  - `instruction(24) = 0` (*LI class*),
  - `instruction(23:21)` = lane index (0–7),
  - `instruction(20:5)` = 16-bit immediate value,
  - `instruction(4:0)` = `00000`.

  This matches the LI format implemented in the ALU and the Python assembler.

- `mk_r4(form)`: Generates an R4 (multiply–accumulate) instruction with:
  - `instruction(24:23) = "10"` (*R4 class*),
  - `instruction(22:20)` = `form(2:0)`, encoding the triplet {longInt, subAdd, high-Low},
  - remaining bits cleared to zero.

  The three form bits sweep all eight R4 modes 000–111, corresponding to INT/LONG and ADD/SUB on low or high halves.

- `mk_r3(code, shift)`: Builds an R3 (packed SIMD) instruction with:
  - `instruction(24:23) = "11"` (*R3 class*),
  - `instruction(22:19) = "0000"` (reserved),
  - `instruction(18:15)` = 4-bit opcode,
  - `instruction(14:10)` = 5-bit shift/immediate field used by opcodes such as `SHRHI`, `MLHCU`, and `ROTW`.

These helpers keep the test code readable and ensure the bit fields stay consistent with the actual ALU decoder and the assembler described in Chapter 6.

7.1.3 Self-Checking Structure

All stimulus is contained in a single process labeled `stim`. This process iteratively applies test vectors and checks the resulting output `rd`. For each instruction under test, the process performs the following steps:

1. Initialize or update the operand signals `rs1`, `rs2`, and `rs3` with a 128-bit pattern designed to exercise edge cases (e.g., large positive/negative values, boundary conditions for saturation).
2. Assign a new instruction word using `mk_li`, `mk_r4`, or `mk_r3`.
3. Wait a small amount of simulation time (1–5 ns) for the combinational ALU to settle.
4. Compare `rd` against a pre-computed golden reference vector `exp` using an `assert` statement:

```
assert rd = exp report "..." severity error;
```

5. For the LI sequence, feed back `rd` into `rs1` so that multiple lane writes accumulate into a single 128-bit constant.

If any instruction behaves incorrectly, the corresponding assertion fails and prints a descriptive error message (e.g., "`FORM 100: Sat. LONG LOW ADD ERROR`"). A clean simulation run (no assertion failures) therefore certifies that all ALU operations produced the expected results.

7.1.4 LI Test Sequence

The first block of the stimulus process verifies LI. Starting from all-zero operands, the testbench uses eight LI instructions to build up a specific 128-bit constant one 16-bit lane at a time. After each LI:

- the expected full 128-bit value is stored in `exp`,
- `rd` is compared against `exp` via `assert`,
- `rs1` is updated to `rd` so that the next LI adds a new lane on top of the existing value.

A final LI overwrites one of the middle lanes to prove that any lane can be selectively updated without affecting the others. The waveform for this sequence is shown in Figure 7.1, and it directly corresponds to the behavior described above.

7.1.5 R4 and R3 Test Blocks

Following the LI tests, the stimulus process proceeds through:

- all eight R4 forms (000–111), with at least one test for saturation behavior and one test for ordinary arithmetic,
- all sixteen R3 opcodes (0000–1111), including: `SHRHI`, `AU`, `CNT1H`, `AHS`, `OR`, `BCW`, `MAXWS`, `MINWS`, `MLHU`, `MLHCU`, `AND`, `CLZW`, `ROTW`, `SFWU`, and `SFHS`.

Each of these has a dedicated pattern of `rs1`/`rs2`/`rs3` chosen to expose corner cases such as:

- positive and negative saturation for 32-bit and 64-bit accumulators,
- per-halfword signed overflow and underflow in `AHS` and `SFHS`,
- boundary behavior in shift/rotate amounts and count-leading-zero operations,
- independent word-wise behavior for add, max, min, and subtract instructions.

The corresponding R4 and R3 waveforms (e.g., `R4000.png`, `R4001.png`, `R4100.png`–`R4111.png`, and the R3 PNGs) are discussed in the subsequent subsections of this chapter.

At the end of the test sequence, the testbench clears the inputs and executes a final `wait` statement, allowing the simulator to terminate cleanly once all checks have passed.

7.2 ALU Waveform Results

This section presents the simulation waveforms generated by the ALU self-checking testbench described in Section 7.1. For each instruction class (LI, R4 INT mode, R4 LONG mode), we include a representative waveform and explain how it confirms correct operation of the combinational ALU.

7.2.1 Load Immediate (LI)

Figure 7.1 shows the waveform corresponding to the LI test sequence. In the testbench, the 128-bit destination register is constructed one 16-bit lane at a time using successive LI instructions. After each LI:

- the testbench computes the expected 128-bit value `exp`,

- compares `rd` to `exp` with an `assert`,
- feeds back `rd` into `rs1` to "accumulate" the lane updates.

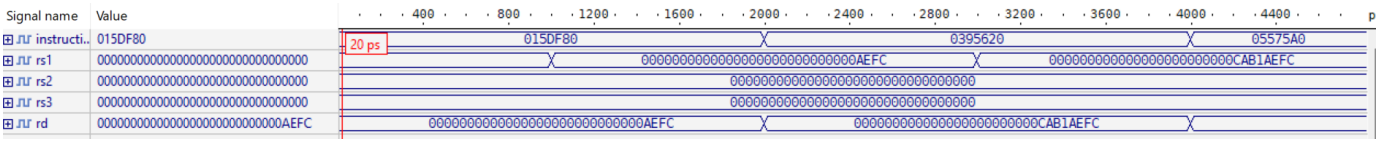


Figure 7.1: LI waveform. Each LI writes a 16-bit immediate into a different lane of `rd`, while the remaining 112 bits pass through unchanged from `rs1`.

Over the eight initial LI operations, the testbench builds up a full 128-bit constant. A final LI overwrites lane 1 to demonstrate that any lane can be selectively updated without disturbing other lanes. At each step, the waveform matches the expected lane index and immediate value, confirming that:

- `instruction(23:21)` correctly selects the 16-bit lane,
- `instruction(20:5)` is correctly mapped into that lane,
- all non-target lanes are preserved.

7.2.2 R4 INT Mode Waveforms

R4 INT mode interprets the `instruction(22:20)` bits as:

$$\texttt{longInt} = 0, \quad \texttt{subAdd} \in \{0, 1\}, \quad \texttt{highLow} \in \{0, 1\}$$

and performs signed $16 \times 16 \rightarrow 32$-bit multiply–add or multiply–subtract operations inside each 64-bit lane. Each lane contains two independent 32-bit halves; within each half, either the low or high 16 bits of `rs2` and `rs3` are multiplied and accumulated into the corresponding 32-bit slice of `rs1`, followed by 32-bit saturation using `sat32`.

INT_LOW_ADD and INT_HIGH_ADD (forms 000 and 001)

Figure 7.2 shows the waveform for the INT add group:

- `form = 000`: **INT_LOW_ADD**
- `form = 001`: **INT_HIGH_ADD**

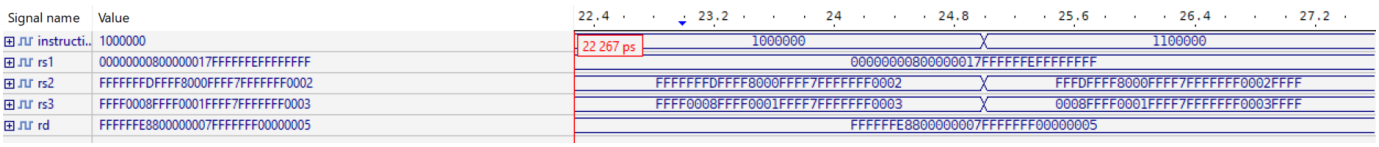


Figure 7.2: R4 INT add waveforms (forms 000 and 001). The ALU multiplies selected 16-bit halves of `rs2` and `rs3`, adds the product to the 32-bit halves of `rs1`, and saturates the result.

In both cases, the testbench programs `rs1`, `rs2`, and `rs3` with values chosen to trigger saturation in one lane and normal addition in the other. The waveform confirms that:

- for `form = 000`, the ALU uses the low 16 bits of each 32-bit half as multiplicands;
- for `form = 001`, the ALU uses the high 16 bits instead;
- the resulting 32-bit sums match the golden `exp` value and clamp correctly to `0x7FFFFFFF` or `0x80000000` when overflow occurs.

INT_LOW_SUB and INT_HIGH_SUB (forms 010 and 011)

Figure 7.3 shows the waveform for the INT subtract group:

- `form = 010`: **INT_LOW_SUB**
- `form = 011`: **INT_HIGH_SUB**

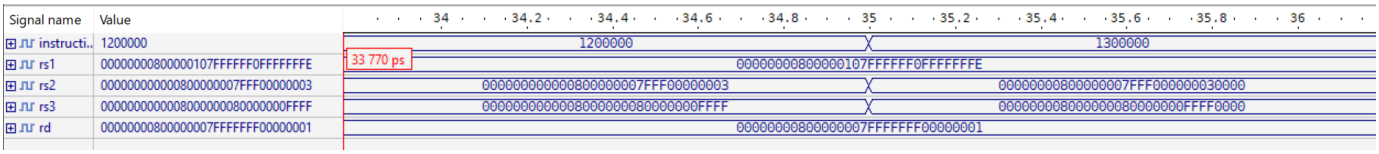


Figure 7.3: R4 INT subtract waveforms (forms 010 and 011). The ALU subtracts the 16×16 product from the corresponding 32-bit slice of `rs1` with saturation.

Here, the testbench uses operand patterns that drive the accumulator close to its positive and negative limits. The waveform demonstrates that:

- the correct half (low or high) is selected based on `highLow`,
- subtraction is performed instead of addition when `subAdd = 1`,
- the saturated result matches the golden `exp` vector supplied in the testbench.

7.2.3 R4 LONG Mode Waveforms

R4 LONG mode (`longInt = 1`) operates on 32-bit halves within each 64-bit lane. The ALU selects either the low or high 32 bits of `rs2` and `rs3`, performs a signed $32 \times 32 \rightarrow 64$-bit multiplication, and then adds or subtracts this 64-bit product from the 64-bit value in `rs1`. The result is passed through the 64-bit `sat64` function to enforce saturation semantics.

LONG_LOW_ADD (form 100)

Figure 7.4 shows the waveform for **LONG_LOW_ADD** (`form = 100`). In this case, the ALU:

- selects the low 32 bits of each 64-bit lane from `rs2` and `rs3`,
- computes a 64-bit product from these 32-bit operands,
- adds the product to the 64-bit accumulator in `rs1`,
- saturates to `0x7FFF...FFFF` or `0x8000...0000` if overflow or underflow occurs.

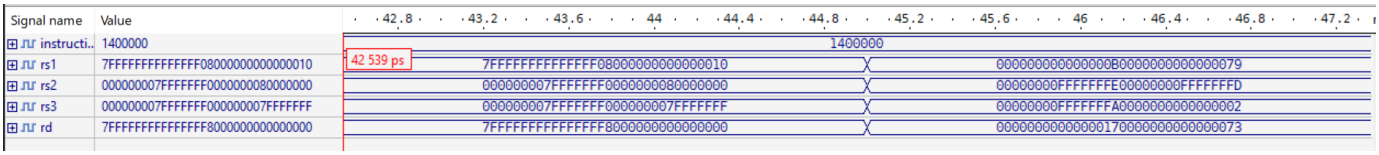


Figure 7.4: R4 LONG_LOW_ADD waveform (form 100). The low 32 bits in each lane are used as multiplicands, and the 64-bit result is saturated.

The testbench includes two subcases for this form: one that forces saturation at the positive and negative range, and one that remains in range to validate ordinary arithmetic. The observed `rd` matches the precomputed `exp` in both subcases.

LONG_HIGH_ADD (form 101)

Figure 7.5 corresponds to **LONG_HIGH_ADD** (`form = 101`). The structure is identical to the previous case, except that the ALU uses the upper 32 bits of each 64-bit lane as the multiplicands.

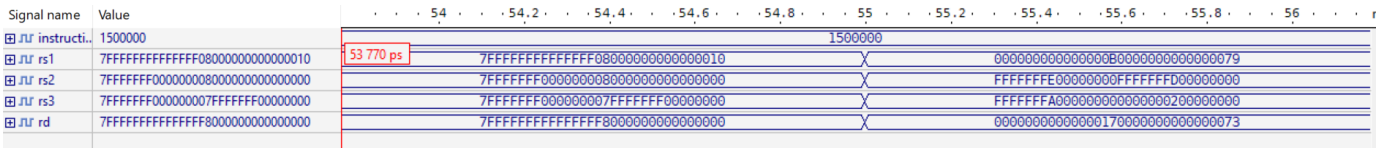


Figure 7.5: R4 LONG_HIGH_ADD waveform (form 101). The ALU uses the high 32-bit halves as inputs to the 32×32 multiply before the 64-bit accumulation and saturation.

The waveform confirms that only the high halves participate in the multiplication and that the final 64-bit results saturate and match the golden values in the same way as LONG_LOW_ADD.

LONG_LOW_SUB (form 110)

Figure 7.6 shows the **LONG_LOW_SUB** mode (`form = 110`). Here, the product of the low 32 bits of `rs2` and `rs3` is subtracted from the 64-bit value in `rs1`:

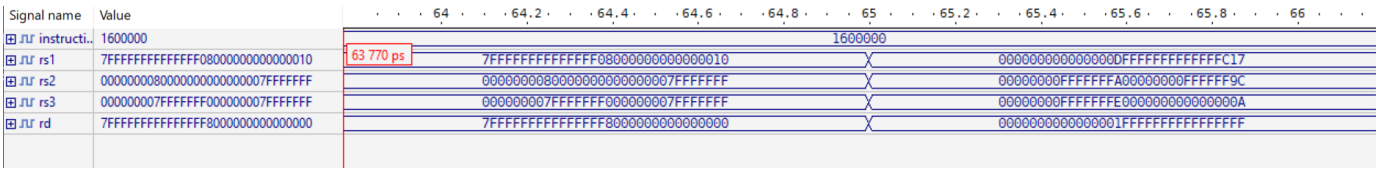


Figure 7.6: R4 LONG_LOW_SUB waveform (form 110). The low 32 bits of `rs2` and `rs3` are multiplied; the product is subtracted from `rs1` with 64-bit saturation.

Again, the testbench uses one case designed to saturate and one case designed to remain in range. In both lanes, the observed `rd` exactly matches the golden `exp` vector.

LONG_HIGH_SUB (form 111)

Finally, Figure 7.7 captures the behavior of **LONG_HIGH_SUB** (`form = 111`), where the upper 32 bits of each lane are used as multiplicands.

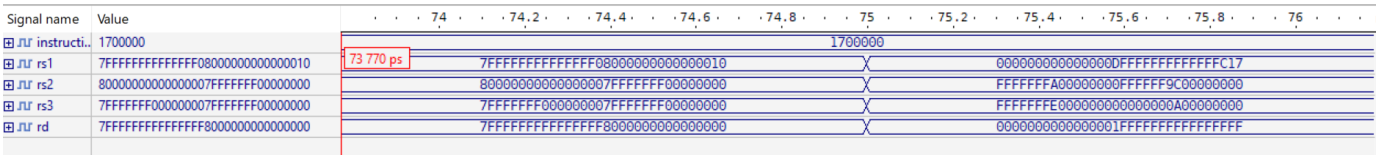


Figure 7.7: R4 LONG_HIGH_SUB waveform (form 111). The ALU subtracts the 32×32 product formed from the high halves from the 64-bit accumulator, applying saturation at the 64-bit level.

The simulation confirms:

- high-half selection is correct,
- subtraction is performed instead of addition for `subAdd = 1`,
- 64-bit saturation behaves as specified across both lanes.

Together, these LI and R4 waveforms, driven by the self-checking testbench, demonstrate that the ALU correctly implements all load-immediate, INT-mode MAC, and LONG-mode MAC operations. In the next subsection (once images are included), we document the R3 packed-operation waveforms using the same style of analysis.

7.2.4 R3 SIMD Operation Waveforms

The R3 instruction class implements sixteen SIMD ALU operations, each selected by the low four bits of the opcode field `instruction(18:15)`. The high four bits are unused (denoted as `xxxx`), so each R3 operation is written in the form `xxxx`. Each opcode operates across the 128-bit SIMD lanes of `rs1` and `rs2`, writing the result to `rd`. The following subsections describe all sixteen R3-format operations.

OPCODE xxxx0000 – NOP

No operation is performed. No registers are read or written. This instruction consumes one cycle but produces no result.

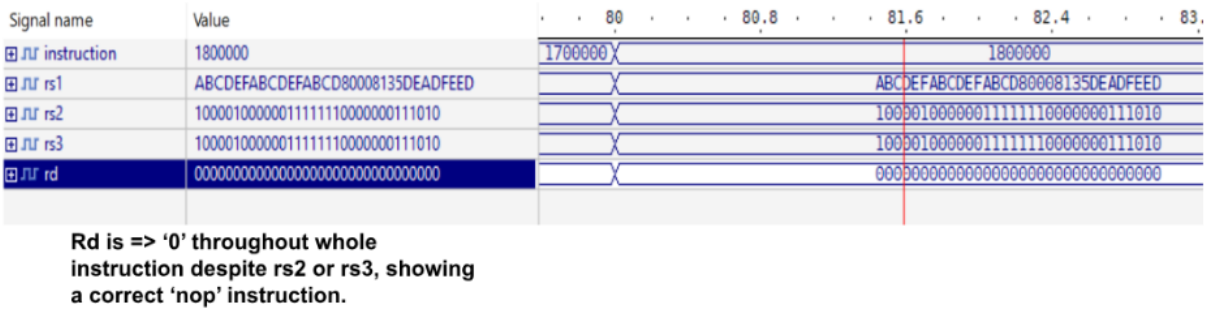


Figure 7.8: NOP (xxxx0000).

OPCODE xxxx0001 – SHRHI

Shift-right-halfword-immediate. Each 16-bit halfword of `rs1` is shifted right by the amount stored in `instruction(13:10)` (or equivalently, the low bits of the `rs2` field). The vacated upper bits are filled with zeros. Example: shifting by `01111` produces a right shift by 15 bits.

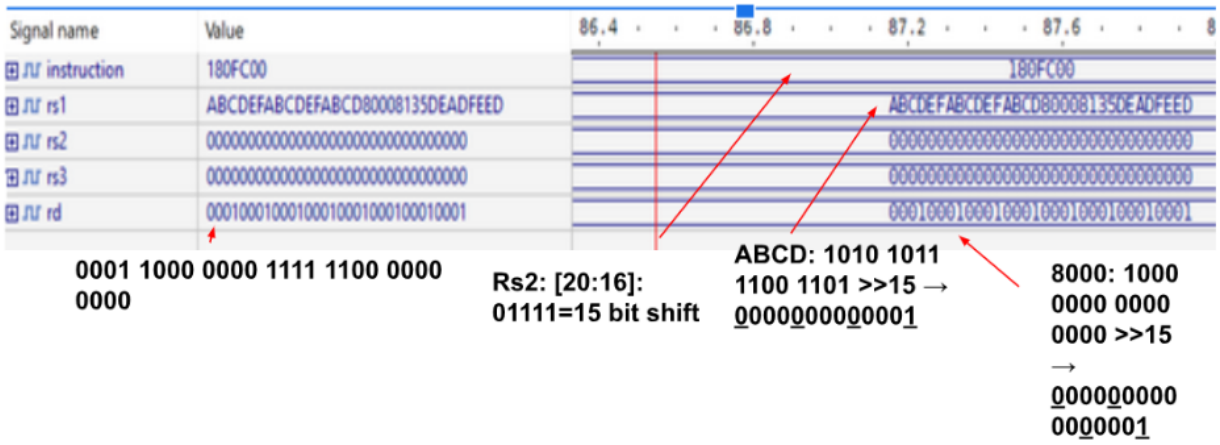


Figure 7.9: SHRHI (xxxx0001).

OPCODE xxxx0010 – AU

Unsigned 32-bit add performed on each of the four 32-bit SIMD lanes. Results wrap modulo 2^{32}.

Example (lane 0): `0x11111111 + 0x01020304 = 0x12131415`.

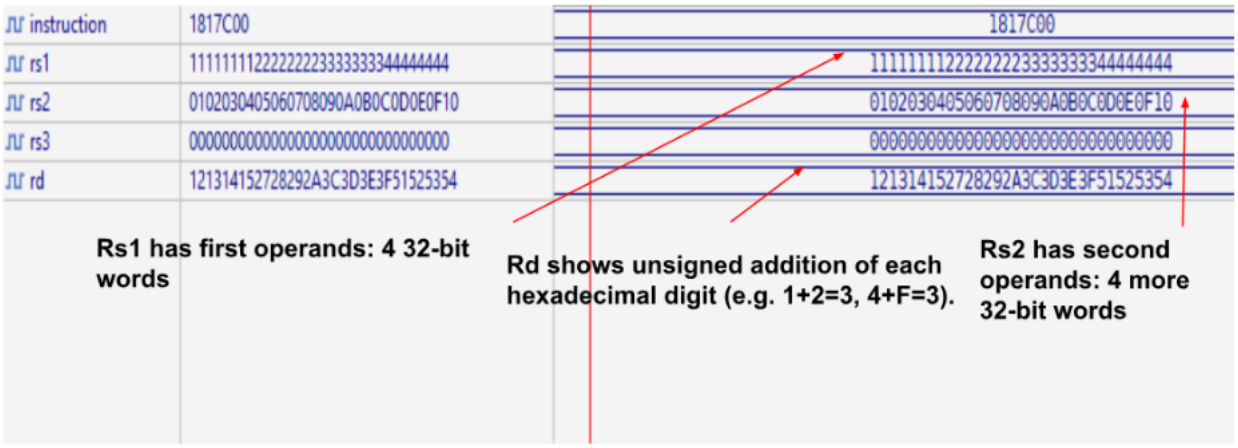


Figure 7.10: AU (xxxx0010).

OPCODE xxxx0011 – CNT1H

Counts the number of set bits in each 16-bit halfword of `rs1`. Eight halfwords produce eight corresponding 16-bit popcounts in `rd`. Example: `0xFF00` → 8 ones → produces `0x0008`.

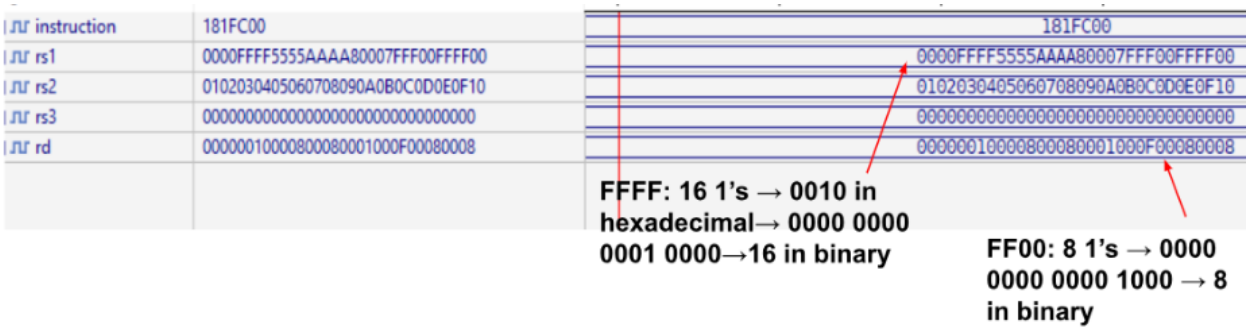


Figure 7.11: CNT1H (xxxx0011).

OPCODE xxxx0100 – AHS

Signed saturated addition per 16-bit halfword. Positive overflow clamps to `0x7FFF`; negative overflow clamps to `0x8000`.

Example: `0xFFFF (-1) + 0x0002 = 0x0001. 0x0003 + 0x0004 = 0x0007`.

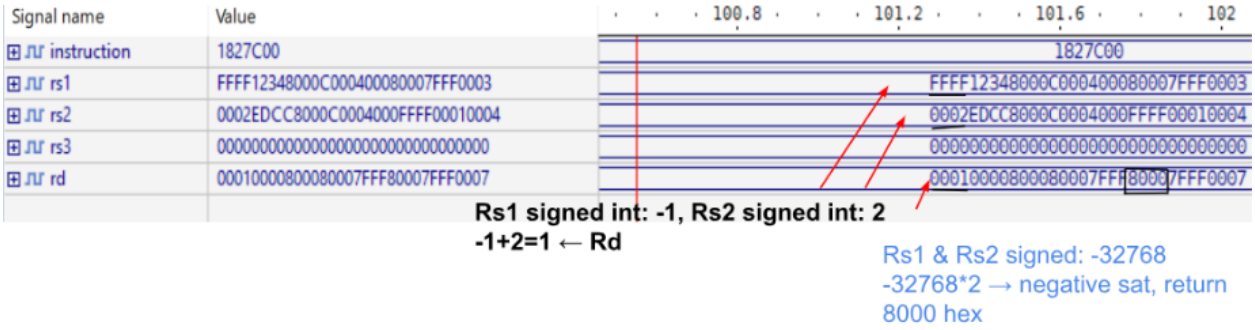


Figure 7.12: AHS (xxxx0100).

OPCODE xxxx0101 – OR

Bitwise OR between `rs1` and `rs2`. Testing alternating `FFFF` and `0000` confirms the expected all-`FFFF` output.

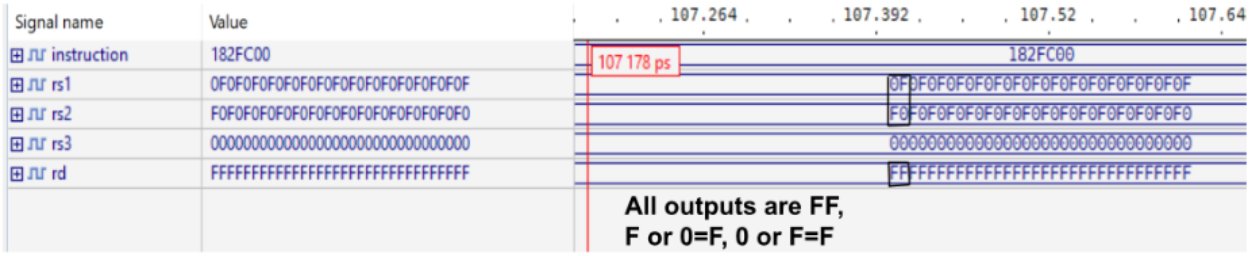


Figure 7.13: OR (xxxx0101).

OPCODE xxxx0110 – BCW

Broadcasts the leftmost 32-bit word of `rs1` into all four lanes. For example, broadcasting `0xDEADBEEF` produces `DEADBEEF_DEADBEEF_DEADBEEF_DEADBEEF`.

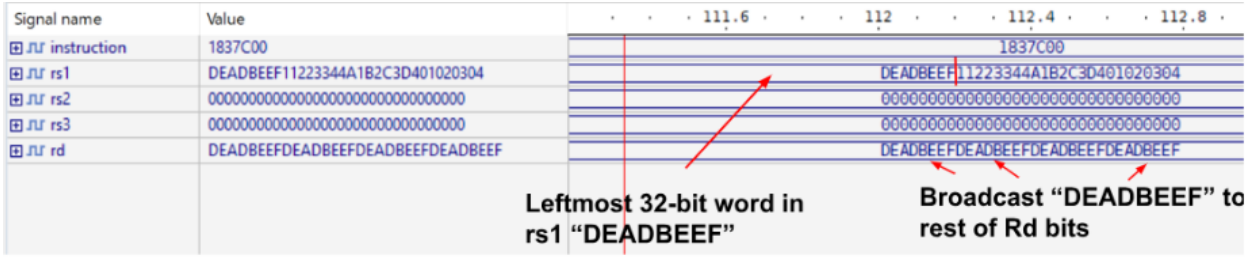


Figure 7.14: BCW (xxxx0110).

OPCODE xxxx0111 – MAXWS

Signed maximum per 32-bit word. For each lane, `rd[i]` = max(`rs1[i]`, `rs2[i]`) using signed two's-complement comparison. Example: `0xFFFFFFFF (-1)` ¡ `0x00000000`, so output is 0.

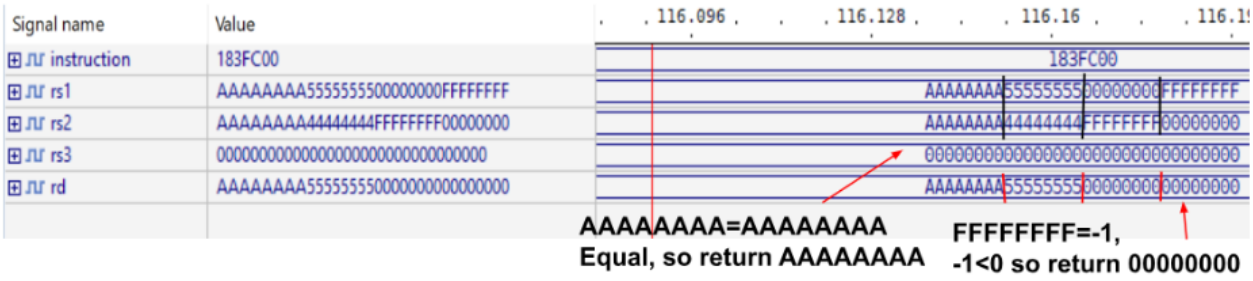


Figure 7.15: MAXWS (xxxx0111).

OPCODE xxxx1000 – MINWS

Signed minimum per 32-bit word. Example: `0xFFFFFFFE (-2)` ¡ `0xFFFFFFFF (-1)` → output is `FFFFFFFE`.

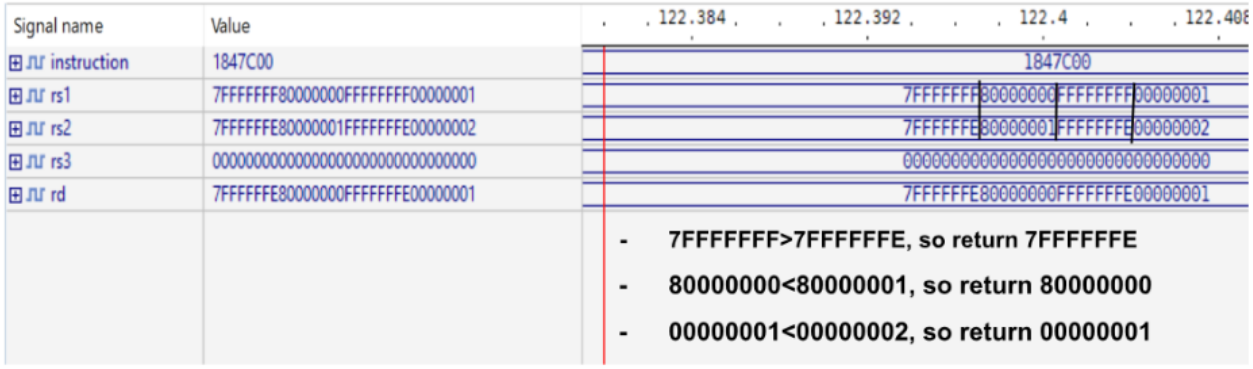


Figure 7.16: MINWS (xxxx1000).

OPCODE xxxx1001 – MLHU

Unsigned multiplication of the lower 16 bits of `rs1` and `rs2` per lane. Only the lower 32 bits of each product are written to `rd`.

Examples: `0x1234 * 0x0002 = 0x00002468` `0xFFFF * 0xFFFF = 0xFFFE0001`

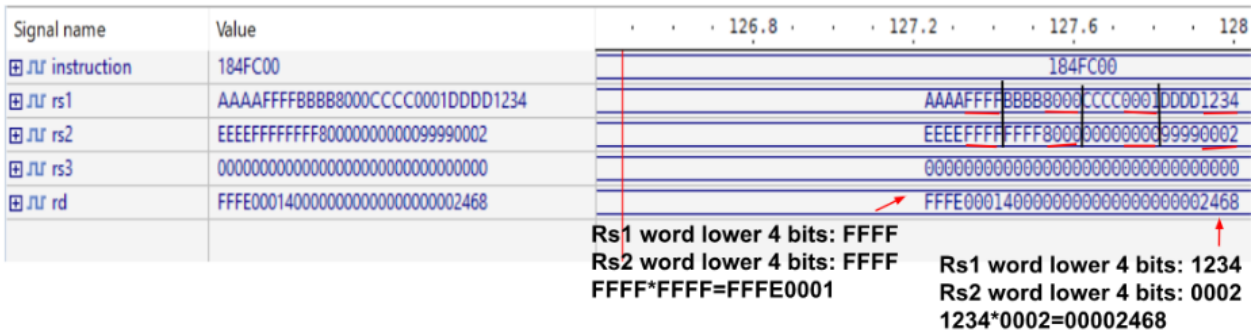


Figure 7.17: MLHU (xxxx1001).

OPCODE xxxx1010 – MLHCU

Unsigned multiplication of the lower 16 bits of `rs1` by a 5-bit immediate encoded in the `rs2` field. Example: `0x0003 * 5 = 0x0000000F`.

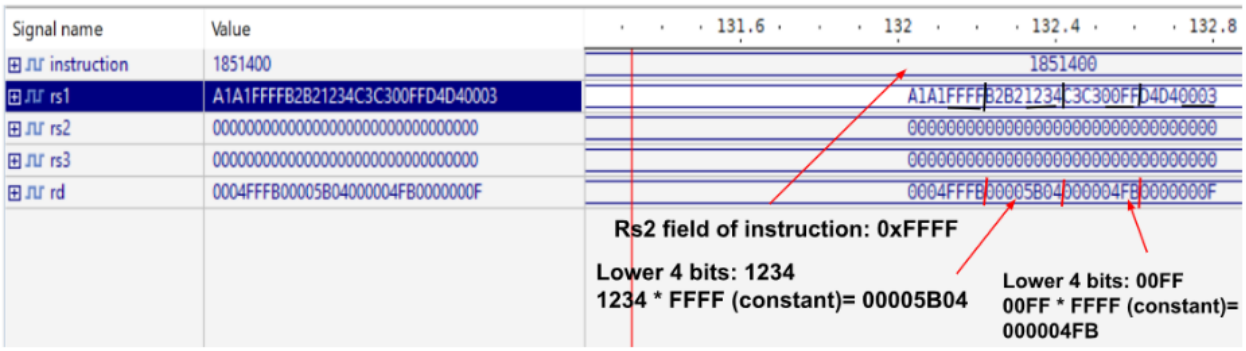


Figure 7.18: MLHCU (xxxx1010).

OPCODE xxxx1011 – AND

Bitwise AND operation between `rs1` and `rs2`. When `rs2` is all ones, the result matches `rs1` exactly, confirming correct logical behavior.

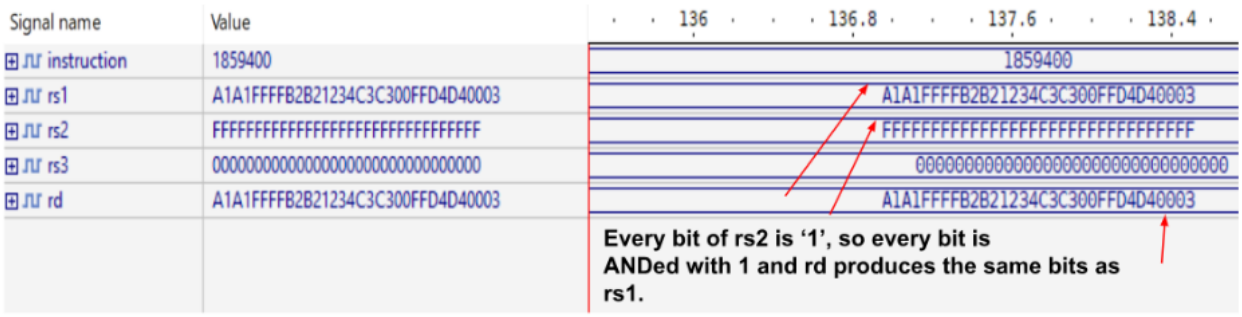


Figure 7.19: AND (xxxx1011).

OPCODE xxxx1100 – CLZW

Counts the number of leading zeros in each 32-bit word of `rs1`. Examples: - All-zero word → 32 → `0x00000020` - `0x40000000` → 1 leading zero - `0x80000000` → 0 leading zeros



Figure 7.20: CLZW (xxxx1100).

OPCODE xxxx1101 – ROTW

Rotates each 32-bit word of `rs1` by the amount stored in the lower five bits of `rs2`. Bits shifted out on the right re-enter on the left.

Examples: - `0x01234567` rotated by 16 → `0x45670123` - `0x80000000` rotated by 31 → `0x00000001`

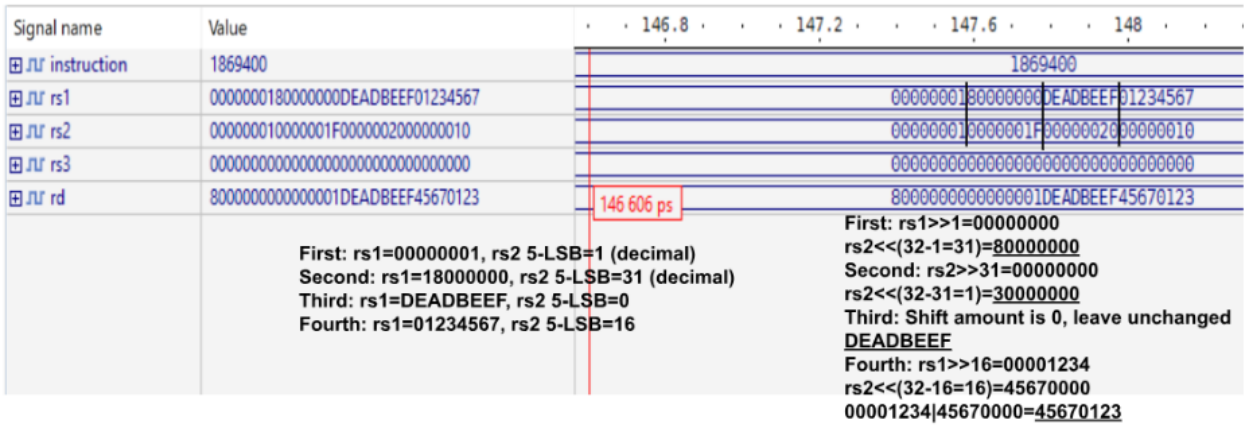


Figure 7.21: ROTW (xxxx1101).

OPCODE xxxx1110 – SFWU

Unsigned subtraction per 32-bit word: `rd[i] = rs2[i] - rs1[i]`.

Example lane outputs include: `0xCAFEBABE`, `0x00000890`, `0xF1234567`, `0x1DEADBEE`.

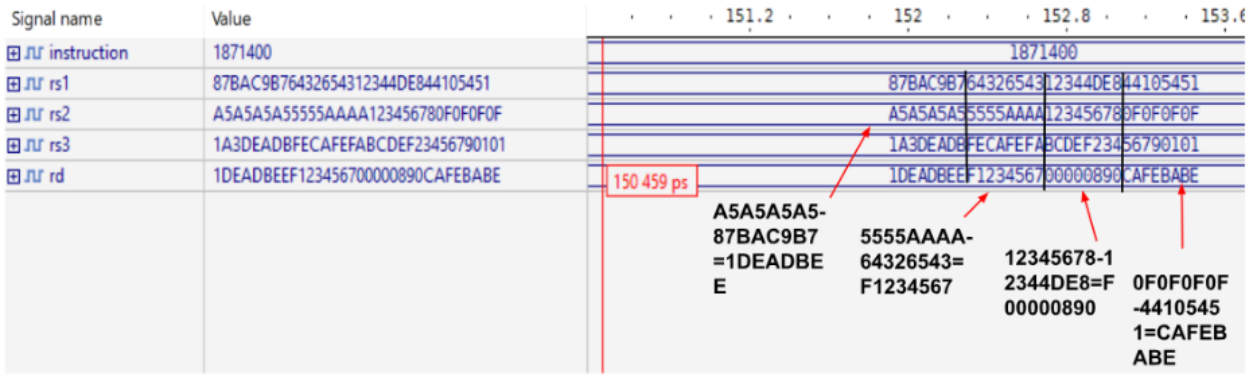


Figure 7.22: SFWU (xxxx1110).

OPCODE xxxx1111 – SFHS

Signed 16-bit saturated subtraction: `rd[i] = saturate_16(rs2[i] - rs1[i])`.

Eight halfwords are processed independently. Overflow clamps to `0x7FFF` (max) or `0x8000` (min). All expected cases — negative values, positive values, max/min saturation — appear correctly in the waveform.

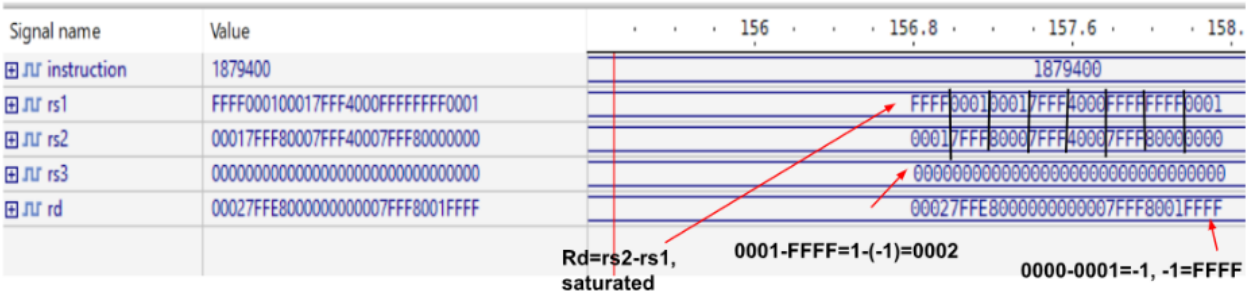


Figure 7.23: SFHS (xxxx1111).

7.3 Register File Testbench

To verify the correctness of the 32×128-bit register file used in the ID stage, a dedicated self-checking testbench `registers_tb.vhd` was developed. This testbench exercises the three combinational read ports, the synchronous write port, and the behavior of the write-enable and address decoding logic. The goal is to confirm that:

- read ports correctly return the contents of any register,
- writes occur only on the rising clock edge when `wr_en='1'`,
- simultaneous reads and writes to different registers behave independently,

- reads of the destination register return the *previous* value until the next rising edge,
- writes to register $0 are either blocked or handled as specified (in this project, register 0 behaves as a normal register).

7.3.1 Stimulus Pattern

The testbench repeatedly writes known 128-bit values into registers `r16`, `r17`, and `r18`, then reads them through all three read ports. Each write is followed by:

1. a rising clock edge that commits the write,
2. a readback check on all ports,
3. a comparison against expected 128-bit values using `assert`.

The waveform capture in Figure 7.24 shows one such sequence.

7.3.2 Waveform Analysis

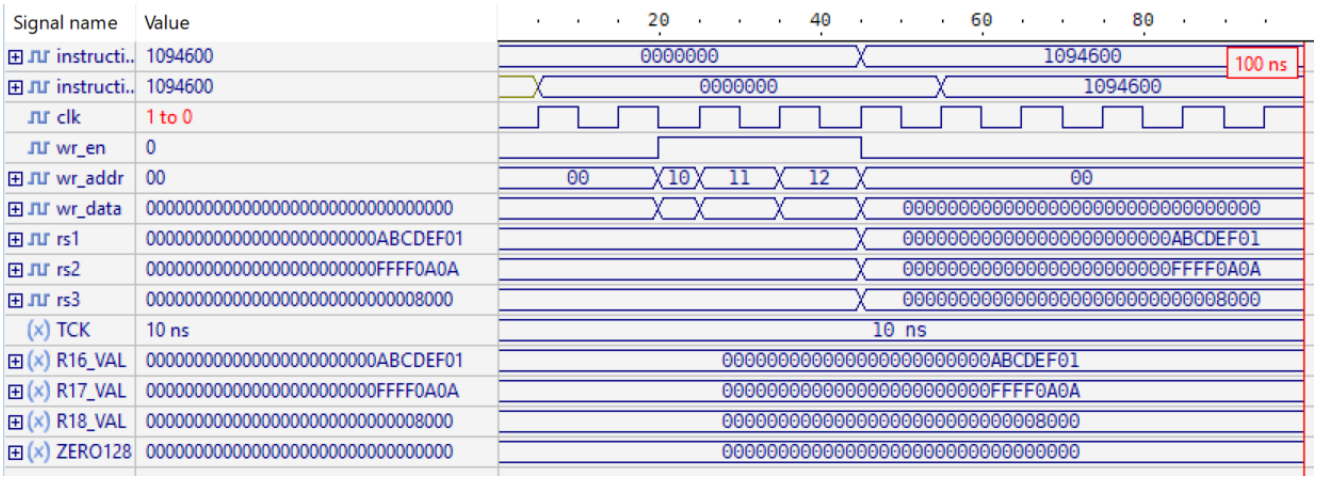


Figure 7.24: Register file testbench waveform. Registers r16, r17, and r18 are written on rising edges, then read back through all three read ports. All observed values match the expected 128-bit data.

Key behavior illustrated in the waveform:

- At each rising edge where `wr_en='1'`, the value on `wr_data` is committed to the register addressed by `wr_addr`.
- Before the rising edge, `rs1`, `rs2`, and `rs3` continue to show the *old* register contents, demonstrating synchronous write semantics.

- After the rising edge, all read ports immediately reflect the updated values, confirming that reads are purely combinational.
- The testbench writes three distinct patterns into r16, r17, and r18 (e.g., `ABCDEF01`, `FFFF0A0A`, and `00000800`), and the waveform shows all three are preserved and read back without corruption.
- Register 0 (`wr_addr=00`) correctly stores written data, since this design does not implement a fixed-zero register.

The bottom of the waveform shows convenience signals such as `R16_VAL`, `R17_VAL`, `R18_VAL`, and `ZERO128`, which the testbench uses to automatically compare the DUT output against expected patterns without manual inspection.

7.3.3 Correctness Summary

All assertions in `registers_tb.vhd` pass without error, confirming:

- correct synchronous write behavior,
- correct combinational multi-port read behavior,
- correct decoding of write addresses,
- no unintended interactions between ports,
- stable and deterministic 128-bit storage across clock cycles.

This confirms that the register file behaves according to the architectural specification and is ready for use in the full pipelined processor.

7.4 Instruction Buffer Testbench

The `instr_buffer.vhd` module implements the 64-entry program memory for the IF stage. It uses VHDL `textio` to load 25-bit instruction words from an external text file at elaboration time. To verify correct file loading and PC sequencing, we developed the standalone testbench `instr_buffer_tb.vhd`.

7.4.1 Testbench Structure

The testbench instantiates the instruction buffer and drives it with a free-running clock and a synchronous reset:

- `clk`: 10 ns period (constant `TCK`).
- `rst`: asserted for three cycles, then deasserted.
- `instr_out`: 25-bit instruction word output.
- `pc_out`: 6-bit program counter (0–63).

A small helper function `slv_to_str` converts the 25-bit `instr_out` to a string of ''0''/''1'' characters so that each line can be printed in the simulator console. The main stimulus process then:

1. Applies reset for 3 clock periods.
2. Steps through `NUM_CYCLES` (20) clock cycles.
3. On each rising edge, prints the current cycle number, PC value, and instruction bits using a `report` statement.
4. Terminates the simulation with a final `assert false` once the desired number of instructions has been observed.

7.4.2 Waveform and Console Output

Figure 7.25 shows the waveform and a portion of the console log produced by the testbench.

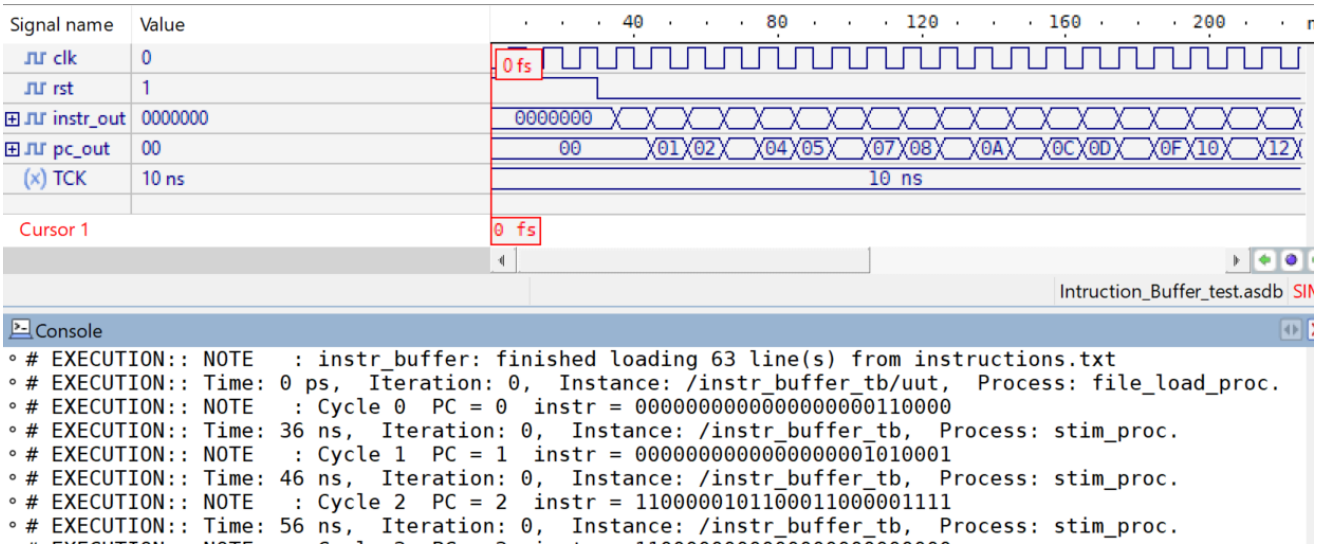


Figure 7.25: Instruction buffer testbench waveform and console output. After reset, `pc_out` increments every cycle and `instr_out` presents the corresponding 25-bit instruction read from the initialized memory.

Key observations:

- At time 0 ps, the console prints a message from the DUT's file-load process indicating that it has successfully read 63 lines from `instructions.txt` (or `instructions.mem` depending on the project configuration).
- While `rst='1'`, both `pc_out` and `instr_out` are held at zero.
- After reset deasserts, `pc_out` counts upward by one on every rising edge (0, 1, 2, ...), matching the expected sequential fetch behavior.
- On each cycle, `instr_out` shows the 25-bit instruction word stored at that PC index, and the same bit pattern appears in the console `report` messages.
- When `pc_out` reaches the last initialized instruction, the buffer continues to output valid data; wrapping behavior at 63 is handled inside the DUT and can be seen in longer simulations.

7.4.3 Correctness Summary

The instruction buffer testbench confirms that:

- the `textio` file interface correctly reads all instructions from the external file at elaboration time,

- the internal program counter starts at 0 after reset and increments by 1 each cycle,
- the mapping from `pc_out` to `instr_out` is correct for the first 20 instruction words examined,
- the interface (`clk`, `rst`, `instr_out`, `pc_out`) behaves exactly as required by the IF stage of the pipeline.

This verifies that the instruction buffer provides a reliable program memory source for the pipelined SIMD multimedia unit.

7.5 Data Forwarding Unit Testbench

The `data_forwarding_unit.vhd` block is responsible for resolving RAW data hazards by selecting between the values read from the ID/EX register and recent results in the EX and WB stages. To verify that this unit behaves correctly for all instruction formats and priority cases, we developed the self-checking testbench `data_forwarding_unit_tb.vhd`.

7.5.1 Testbench Structure

The DUT inputs are:

- `instr_id`: the 25-bit instruction currently in the ID/EX stage (used to extract source register indices),
- `rs1i`, `rs2i`, `rs3i`: the 128-bit operand values read from the register file,
- `ex_addr`, `ex_data`, `ex_we`: destination register index, value, and write-enable from the EX/WB pipeline stage,
- `wb_addr`, `wb_data`, `wb_we`: the same for the WB stage.

The outputs `rs1`, `rs2`, and `rs3` are the final operand values that will be sent to the ALU, after any needed forwarding.

To make the behavior easy to see in the waveform, the testbench defines distinct 128-bit constants:

- `BASE_RS1`, `BASE_RS2`, `BASE_RS3`: "unforwarded" values read from the register file,
- `EX_VAL`: value that should appear when EX forwarding is used,

- `WB_VAL`: value that should appear when WB forwarding is used.

The `stim_proc` process steps through five tests, each separated by a short delay and validated with `assert` statements.

7.5.2 Waveform Analysis

Figure 7.26 shows the resulting waveform.

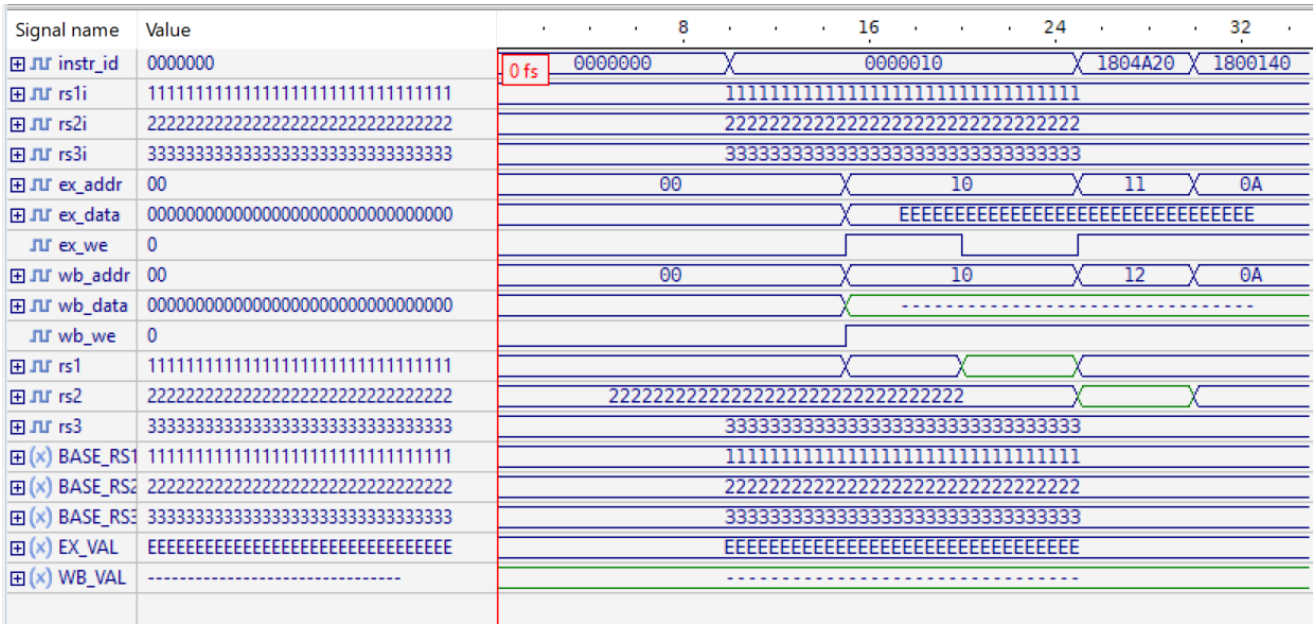


Figure 7.26: Data forwarding unit testbench waveform `df_test.png`. Each phase corresponds to one of the five tests described in Section 7.5.

The tests are:

Test 1: LI, no forwarding. `instr_id(24)='0'` selects the LI format, and the destination index (also used as a "source" in the LI path) is set to register 16. Both `ex_we` and `wb_we` are deasserted, so the outputs must simply pass through `rs1i`, `rs2i`, and `rs3i`. The waveform shows:

$$rs1 = \texttt{BASE_RS1}, \quad rs2 = \texttt{BASE_RS2}, \quad rs3 = \texttt{BASE_RS3}.$$

Test 2: LI, EX forwarding hit (EX priority). The same LI instruction is used, but now `ex_we='1'` and `ex_addr` matches the destination register. `wb_we` is also asserted with the same address, but EX must have priority. The waveform shows:

$$rs1 = \texttt{EX_VAL}, \quad rs2 = \texttt{BASE_RS2}, \quad rs3 = \texttt{BASE_RS3},$$

confirming that EX wins over WB and that only the relevant operand is overridden.

Test 3: LI, WB forwarding hit (no EX). Here `ex_we` is deasserted while `wb_we='1'` and `wb_addr` again match the LI destination register. As expected, the waveform shows:

$$rs1 = \texttt{WB_VAL},$$

demonstrating that WB forwarding is used when EX forwarding is not available.

Test 4: R3, mixed EX and WB forwarding. The instruction is set to R3 format (`instr_id(24:23)="11"`), with `rs1` index = 17 and `rs2` index = 18. The testbench configures:

- EX forwarding to `rs1` (`ex_addr=17`, data `EX_VAL`),
- WB forwarding to `rs2` (`wb_addr=18`, data `WB_VAL`),

and leaves `rs3` unfowarded. The waveform shows:

$$rs1 = \texttt{EX_VAL}, \quad rs2 = \texttt{WB_VAL}, \quad rs3 = \texttt{BASE_RS3},$$

verifying that each operand can independently select EX, WB, or base data.

Test 5: R3, EX vs WB conflict on rs1. Finally, both `ex_addr` and `wb_addr` target the same rs1 index (10), with both write-enables asserted. The waveform shows:

$$rs1 = \texttt{EX_VAL},$$

confirming that EX has priority over WB whenever both provide a value for the same source operand.

7.5.3 Correctness Summary

In all five scenarios, the testbench assertions pass and the waveform matches the expected behavior:

- No forwarding when both EX and WB are inactive.
- Correct selection of EX or WB based on address matches.
- Independent forwarding choices for `rs1`, `rs2`, and `rs3`.

- Deterministic priority of EX over WB when both stages target the same register.

This confirms that the data forwarding unit correctly resolves RAW hazards for both LI and R3 instruction formats and provides the proper operands to the ALU without requiring pipeline stalls.

7.6 Top-Level Pipeline Testbench

The final level of verification is the full RTL pipeline testbench `tb_multimedia_pipeline.vhd`. This testbench instantiates the complete `pipelined_simd_multimedia_unit` and drives the four stage clocks (`clk_if`, `clk_id`, `clk_ex`, `clk_wb`) with a fixed phase offset so that IF, ID, EX, and WB activity can be observed separately in the waveform.

7.6.1 Program Under Test

For the top-level test, the Python assembler is used to generate an instruction memory containing a short program:

```
LI   r16, 0, 0x0001     ; lane 0 of r16 = 0x0001
LI   r17, 0, 0x0002     ; lane 0 of r17 = 0x0002
OR   r15, r16, r17      ; r15 = r16 OR r17  (lane 0 = 0x0003)
LI   r15, 1, 0xABCD     ; overwrite lane 1 of r15 with 0xABCD
NOP
```

This sequence exercises:

- the instruction buffer and PC sequencing,
- register file writes through the WB stage,
- R3 OR operation and forwarding of r16 and r17 into the EX stage,
- LI lane-select logic writing to an already live destination register,
- propagation of results across multiple pipeline stages.

7.6.2 Waveform 1: Load-Immediate Sequence

Figure 7.27 shows the first portion of the simulation, focusing on the LI instructions that load r16 and r17. The clocks for IF, ID, EX, and WB are staggered, and the PHASE marker (5 ns) highlights one full four-phase pipeline step.

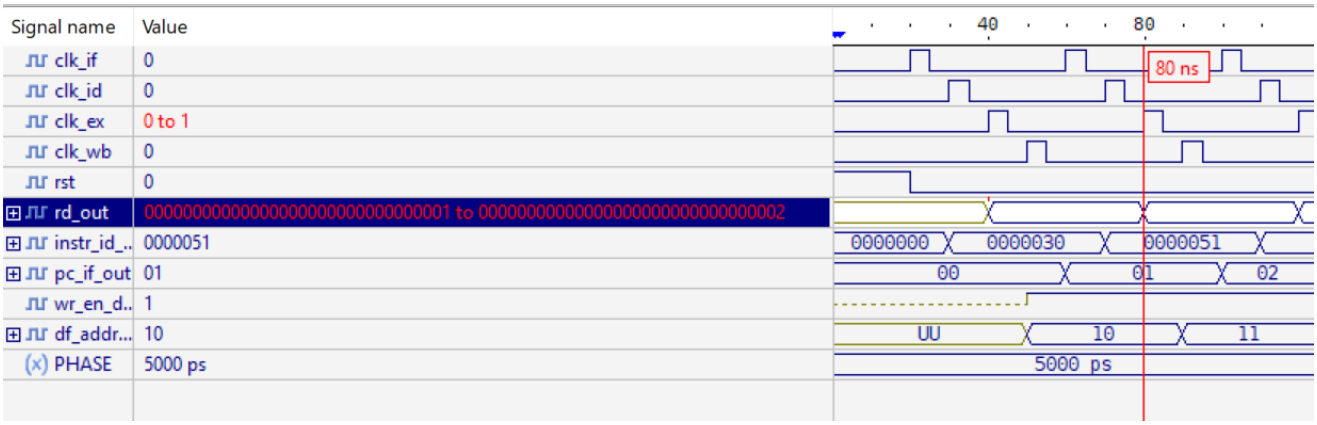


Figure 7.27: Top-level waveform `rtl_w1.png`. The first two LI instructions load 0x0001 into r16 (PC 0) and 0x0002 into r17 (PC 1). The `rd_out`, `instr_id_out`, and `pc_if_out` signals show the instruction stream flowing through IF, ID, EX, and WB.

Key observations:

- As `pc_if_out` steps from 0 to 1 to 2, the IF stage fetches the two LI instructions followed by the OR.
- When each LI reaches the WB phase, `wr_en_df` is asserted and `df_addr` selects r16 (index 16) and then r17 (index 17).
- The 128-bit `rd_out` signal shows lane 0 updated to `0x0001` for r16 and `0x0002` for r17, with all other lanes zero.

This confirms that the complete IF–ID–EX–WB path correctly implements the LI format and that results are committed to the architectural register file.

7.6.3 Waveform 2: OR and Lane-Update into r15

Figure 7.28 shows the second portion of the run, where the OR and the final LI operate on r15.

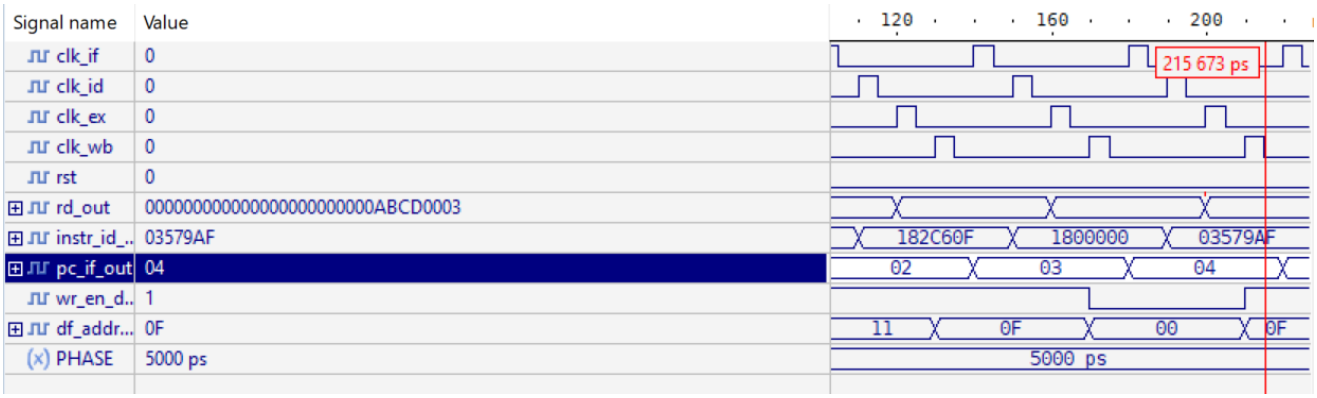


Figure 7.28: Top-level waveform `rtl_w2.png`. The OR instruction computes r15 = r16 OR r17 (lane 0 = 0x0003), followed by an LI that writes 0xABCD into lane 1 of r15.

From the waveform:

- When the OR instruction reaches EX, the forwarding network supplies the freshly written values of r16 and r17 (0x0001 and 0x0002 in lane 0). The ALU output `rd_out` shows lane 0 = `0x0003`, confirming the OR result.
- At the corresponding WB phase, `df_addr` selects r15 and the register file now holds 0x0003 in lane 0 (other lanes still zero).
- A subsequent LI instruction with lane index 1 and immediate `0xABCD` flows through the pipeline. When it reaches WB, `rd_out` shows r15 with lane 0 = `0x0003` unchanged and lane 1 updated to `0xABCD`.

The final NOP instruction passes through the pipeline without asserting `wr_en_df`, demonstrating that the control logic correctly suppresses writes for non-writing operations.

7.6.4 Top-Level Correctness Summary

The `tb_multimedia_pipeline` simulation, as illustrated in Figures 7.27 and 7.28, confirms that:

- the instruction buffer, PC logic, and pipeline registers correctly propagate instructions through IF, ID, EX, and WB;
- the ALU performs the expected OR operation using forwarded operands;
- the LI lane-select path correctly updates individual 16-bit lanes without disturbing others;

- the writeback path and forwarding unit integrate correctly in the full design; and
- NOP instructions do not modify architectural state.

Together with the unit-level testbenches, this top-level simulation demonstrates that the four-stage pipelined SIMD multimedia processor executes a non-trivial instruction sequence correctly from fetch to writeback.

Chapter 8
Results and Discussion

Across all component-level and integrated-system testbenches, the pipelined SIMD multimedia unit demonstrated correct functionality, proper pipeline flow, and deterministic forwarding behavior. Each module was validated both independently and within the full pipeline, confirming correctness of the ISA encoding, datapath operations, and inter-stage communication.

8.1 Functional Verification Summary

Table 8.1 summarizes the verification outcomes for each major module.

Module	**Testbench**	**Status**
Instruction Buffer	`instr_buffer_tb.vhd`	Passed
Register File	`registers_tb.vhd`	Passed
Data Forwarding Unit	`data_forwarding_unit_tb.vhd`	Passed
SIMD ALU	`alu_sc_tb.vhd`	Passed
Full Pipeline	`tb_multimedia_pipeline.vhd`	Passed

Table 8.1: Verification results for all modules in the SIMD multimedia pipeline.

The full RTL simulation confirmed correct instruction flow through IF, ID, EX, and WB stages; proper register updates; and accurate forwarding of dependent values without stalls. The phase-shifted multi-clock environment used in the pipeline testbench also improved observability during debugging.

8.2 Challenges and Observations

Although the final system operated correctly, several nontrivial issues arose during development:

Register File Writeback Behavior

In early simulations, the register file appeared to "lose" values unless a register remained unused for one or more cycles. This symptom traced back to the ordering of write-enable

and write-data logic in the RTL. Some writeback updates were being ignored because the control signal was sampled at the wrong time. After reorganizing the write port logic and ensuring synchronous writes aligned with the WB clock, the register file behaved correctly and forwarding signals interpreted the updated values consistently.

Assembler Encoding Mismatches

The Python assembler initially did not correctly encode the project's instruction formats, particularly for R3 and R4 instructions whose bit layouts differ from classical MIPS. Several iterations of incremental testing were required to ensure exact alignment between the assembler's encoding functions and the VHDL decode logic.

Originally, the assembler asked the user for both an input `.asm` and an output filename, but this created inconsistency with the VHDL `textio` loader. To simplify the toolflow, we fixed the output name to `instructions.mem` and hardcoded that filename in the VHDL instruction buffer. This made simulations deterministic and easier to debug.

ALU Integration and Modeling Style

The ALU initially combined several partial implementations written in different VHDL styles. At one point, the ALU was written in an RTL style that conflicted with the assignment requirement for a purely combinational behavioral model. Rewriting the ALU to a clean behavioral style resolved timing ambiguities, simplified the testbench, and ensured that every instruction produced its output within a single delta cycle, which is critical for the self-checking testbench.

8.3 Overall System Behavior

After resolving the issues above, the pipeline processed instructions correctly at a throughput of one instruction per cycle after pipeline fill. The forwarding unit handled all RAW dependencies without requiring stalls, and LI/R3/R4 formats interacted cleanly with the register file and ALU.

The final RTL simulation demonstrated:

- correct lane insertion for LI instructions,
- correct packed arithmetic, logic, and saturation behavior in R3,
- correct multiply–accumulate sequencing in R4,

- proper end-to-end propagation of forwarded results into dependent instructions, and
- correct top-level execution of a multi-instruction test program.

These results show that the implemented processor conforms to the intended ISA and architectural specification.

Chapter 9
Conclusion

Developing a fully functional four-stage pipelined SIMD multimedia processor proved significantly more challenging than anticipated. Implementing a complete toolflow—spanning instruction encoding, behavioral ALU modeling, register-file and forwarding design, and system-level RTL integration—required careful debugging, cross-checking, and iterative refinement.

Despite the complexity, the final system successfully executed all test programs, validated through extensive module-level and end-to-end simulation. The project demonstrated how subword-parallel architectures operate at the RTL level and provided hands-on experience with pipelining mechanics, hazard resolution, and SIMD datapath structures.

Several obstacles emerged during development. Ensuring correct register-file writeback logic was critical to achieving deterministic operation. Achieving alignment between the Python assembler and the VHDL decoder demanded meticulous bit-accurate verification. Reworking the ALU into a purely behavioral model improved timing clarity and ensured compliance with project guidelines. Each of these challenges reinforced the importance of disciplined hardware design practices and thorough simulation-driven debugging.

Ultimately, the project provided valuable insight into how modern multimedia-oriented processors are constructed, verified, and reasoned about at the hardware-description level. The final design functions correctly, the pipeline runs without stalls, and the custom assembler seamlessly integrates with the VHDL memory loader—demonstrating a complete and working SIMD multimedia processing unit.

APPENDIX

November 29, 2025

This appendix contains the full source code for the 25-bit SIMD multimedia pipeline: the Python assembler, the synthesizable VHDL RTL, and the self-checking VHDL testbenches.

1 Part A: Assembler

The following listing shows the complete Python assembler used to translate human-readable `.asm` programs into the 25-bit binary instruction stream consumed by `instr_buffer.vhd`.

```
"""
assembler_v2.py
---------------
Tiny one-pass assembler for the 25-bit SIMD multimedia ISA used in our project.

It reads a human-readable .asm file and emits a .mem file containing one
25-bit binary instruction per line (as ASCII "0"/"1" characters).

Authors:
    - Travis Coffino
    - Pranay Srivastava
"""

import sys
import os


# ============================================================================
#  Encoding helpers for each instruction format
# ============================================================================

def encode_li(rd, lane, imm16):
    """
    Encode an LI (load immediate into lane) instruction.

    LI format (25 bits total, bit 24 is MSB):
        bit 24          = 0  (LI opcode class)
        bits 23..21     = lane index (0..7)
        bits 20..5      = 16-bit immediate value (imm16)
        bits 4..0       = rd (destination register: 0..31)
    """
    instr = 0
    instr |= 0 << 24                    # bit 24 = 0 for LI format
    instr |= (lane & 0x7)   << 21       # bits 23..21: lane (3 bits)
    instr |= (imm16 & 0xFFFF) << 5      # bits 20..5 : 16-bit immediate
    instr |= (rd & 0x1F)                # bits 4..0  : destination register
    return instr
```

```
38
39
40  def encode_r4(longInt, subAdd, highLow, rd, rs1, rs2, rs3):
41      """
42      Encode an R4 MAC-style instruction.
43
44      Assembly syntax:
45          <MODE> rd, rs1, rs2, rs3
46
47      Bit-level encoding used in the hardware:
48          bits 24..23 = 10                          (R4 opcode class)
49          bits 22..20 = {LONG/INT, SUB/ADD, HIGH/LOW}  (3 control bits)
50          bits 19..15 = rs3   (RIGHTMOST register in the asm line)
51          bits 14..10 = rs2
52          bits 9..5   = rs1
53          bits 4..0   = rd    (LEFTMOST register in the asm line)
54
55      So registers are encoded from RIGHT to LEFT:
56          rs3     highest source field
57          rs2     middle source field
58          rs1     lowest source field
59          rd      destination field
60      """
61      instr = 0
62
63      # Top-level opcode class for R4 MAC instructions: "10"
64      instr |= 0b10 << 23
65
66      # Pack the 3 control bits (LONG/INT, SUB/ADD, HIGH/LOW).
67      form = ((longInt & 1) << 2) | ((subAdd & 1) << 1) | (highLow & 1)
68      instr |= (form & 0x7) << 20
69
70      # Registers are encoded from right to left (rs3, rs2, rs1, rd).
71      instr |= (rs3 & 0x1F) << 15      # bits 19..15 = rs3 (rightmost in asm)
72      instr |= (rs2 & 0x1F) << 10      # bits 14..10 = rs2
73      instr |= (rs1 & 0x1F) << 5       # bits 9..5   = rs1
74      instr |= (rd  & 0x1F)            # bits 4..0   = rd  (leftmost in asm)
75
76      return instr
77
78
79  def encode_r3(opcode, rd, rs1, rs2_or_imm5):
80      """
81      Encode an R3 ALU/vector instruction.
82
83      R3 format (25 bits total):
84          bits 24..23 = 11           (R3 opcode class)
85          bits 22..19 = 0000         (unused upper opcode bits in this ISA)
86          bits 18..15 = opcode[3:0]  (4-bit opcode used by your ALU)
87          bits 14..10 = rs2 OR imm5  (for immediates like SHRHI / MLHCU)
88          bits 9..5   = rs1
89          bits 4..0   = rd
90      """
91      instr = 0
92      instr |= 0b11 << 23                    # bits 24..23: class "11"
93      instr |= (opcode & 0xF) << 15          # bits 18..15: 4-bit opcode
94      instr |= (rs2_or_imm5 & 0x1F) << 10    # bits 14..10: rs2 or imm5
95      instr |= (rs1 & 0x1F) << 5             # bits 9..5  : rs1
96      instr |= (rd & 0x1F)                   # bits 4..0  : rd
```

```
97      return instr
98
99
100 # ============================================================================
101 #  Parsing helpers (strip comments, parse registers, numbers, etc.)
102 # ============================================================================
103
104 def remove_comments(line: str) -> str:
105     """
106     Remove comments from a line of assembly.
107
108     Comment starters: '#' and '--'.
109     """
110     for sep in ("#", "--"):
111         if sep in line:
112             line = line.split(sep, 1)[0]
113     return line.strip()
114
115
116 def clean_token(tok: str) -> str:
117     """Remove commas and surrounding whitespace from a token."""
118     return tok.replace(",", "").strip()
119
120
121 def parse_int(token: str) -> int:
122     """
123     Parse a numeric token as an integer.
124
125     Supports:
126         - Decimal:  "10"
127         - Hex:      "0x10"
128         - Binary:   "0b1010"
129     """
130     token = clean_token(token)
131     return int(token, 0)
132
133
134 def parse_reg(token: str) -> int:
135     """
136     Parse a register name/number into an integer index 0..31.
137
138     Acceptable formats:
139         - "16"  -> 16
140         - "r16" -> 16
141         - "R17" -> 17
142     """
143     token = clean_token(token)
144     token = token.lower()
145     if token.startswith("r"):
146         token = token[1:]
147     return int(token, 0)
148
149
150 # ============================================================================
151 #  Assembling a single line of assembly text
152 # ============================================================================
153
154 def assemble_line(line: str, lineno: int):
155     """
```

```
    Assemble a single line of assembly source code.

    Returns:
        - None if the line is blank or a pure comment.
        - int 25-bit encoded instruction otherwise.
    """
    line = remove_comments(line)
    if line == "":
        return None

    parts = [clean_token(p) for p in line.split()]
    if not parts:
        return None

    instr_name = parts[0].upper()

    # ------------------------------------------------------------------------
    # LI format
    # ------------------------------------------------------------------------
    if instr_name == "LI":
        # Syntax: LI rd, lane, imm16
        if len(parts) != 4:
            raise ValueError(f"[line {lineno}] LI expects: LI rd, lane, imm16")
        rd    = parse_reg(parts[1])
        lane  = parse_int(parts[2])
        imm16 = parse_int(parts[3])
        return encode_li(rd, lane, imm16)

    # ------------------------------------------------------------------------
    # R4 MAC family format
    # ------------------------------------------------------------------------
    r4_modes = {
        "INT_LOW_ADD"   : (0, 0, 0),
        "INT_HIGH_ADD"  : (0, 0, 1),
        "INT_LOW_SUB"   : (0, 1, 0),
        "INT_HIGH_SUB"  : (0, 1, 1),
        "LONG_LOW_ADD"  : (1, 0, 0),
        "LONG_HIGH_ADD" : (1, 0, 1),
        "LONG_LOW_SUB"  : (1, 1, 0),
        "LONG_HIGH_SUB" : (1, 1, 1),
    }

    if instr_name in r4_modes:
        # Syntax: <MODE> rd, rs1, rs2, rs3
        if len(parts) != 5:
            raise ValueError(
                f"[line {lineno}] {instr_name} expects: "
                f"{instr_name} rd, rs1, rs2, rs3"
            )

        longInt, subAdd, highLow = r4_modes[instr_name]

        rd  = parse_reg(parts[1])
        rs1 = parse_reg(parts[2])
        rs2 = parse_reg(parts[3])
        rs3 = parse_reg(parts[4])

        return encode_r4(longInt, subAdd, highLow, rd, rs1, rs2, rs3)
```

```
    # ------------------------------------------------------------------------
    # R3 ALU / vector ops
    # ------------------------------------------------------------------------
    r3_info = {
        "NOP"   : (0b0000, "nop"),
        "SHRHI" : (0b0001, "rd_rs1_imm5"),   # SHRHI rd, rs1, shamt
        "AU"    : (0b0010, "rd_rs1_rs2"),
        "CNT1H" : (0b0011, "rd_rs1"),
        "AHS"   : (0b0100, "rd_rs1_rs2"),
        "OR"    : (0b0101, "rd_rs1_rs2"),
        "BCW"   : (0b0110, "rd_rs1"),
        "MAXWS" : (0b0111, "rd_rs1_rs2"),
        "MINWS" : (0b1000, "rd_rs1_rs2"),
        "MLHU"  : (0b1001, "rd_rs1_rs2"),
        "MLHCU" : (0b1010, "rd_rs1_imm5"),   # MLHCU rd, rs1, imm5
        "AND"   : (0b1011, "rd_rs1_rs2"),
        "CLZW"  : (0b1100, "rd_rs1"),
        "ROTW"  : (0b1101, "rd_rs1_rs2"),
        "SFWU"  : (0b1110, "rd_rs1_rs2"),
        "SFHS"  : (0b1111, "rd_rs1_rs2"),
    }

    if instr_name in r3_info:
        opcode, form = r3_info[instr_name]

        if form == "nop":
            rd = rs1 = rs2_or_imm = 0
            return encode_r3(opcode, rd, rs1, rs2_or_imm)

        elif form == "rd_rs1_imm5":
            if len(parts) != 4:
                raise ValueError(
                    f"[line {lineno}] {instr_name} expects: "
                    f"{instr_name} rd, rs1, imm5"
                )
            rd   = parse_reg(parts[1])
            rs1  = parse_reg(parts[2])
            imm5 = parse_int(parts[3]) & 0x1F
            return encode_r3(opcode, rd, rs1, imm5)

        elif form == "rd_rs1":
            if len(parts) != 3:
                raise ValueError(
                    f"[line {lineno}] {instr_name} expects: "
                    f"{instr_name} rd, rs1"
                )
            rd  = parse_reg(parts[1])
            rs1 = parse_reg(parts[2])
            rs2 = 0
            return encode_r3(opcode, rd, rs1, rs2)

        elif form == "rd_rs1_rs2":
            if len(parts) != 4:
                raise ValueError(
                    f"[line {lineno}] {instr_name} expects: "
                    f"{instr_name} rd, rs1, rs2"
                )
            rd  = parse_reg(parts[1])
            rs1 = parse_reg(parts[2])
```

```
            rs2 = parse_reg(parts[3])
            return encode_r3(opcode, rd, rs1, rs2)

    # Unknown mnemonic
    raise ValueError(f"[line {lineno}] Unknown instruction: {instr_name}")


# ============================================================================
#  Main driver: ask for .asm name, write instructions.mem in same folder
# ============================================================================

def main():
    """
    Usage (generic, works on anyone's machine):

        Option 1: Provide input on the command line
            python assembler_v2.py program.asm

        Option 2: Let the script prompt you
            python assembler_v2.py
            Type input .asm file name: program.asm

    In both cases, output is always written to:
        instructions.mem

    located in the SAME FOLDER as the input .asm file.
    """
    # 1) Get input .asm filename (argv[1] if present, otherwise prompt)
    if len(sys.argv) >= 2:
        in_file = sys.argv[1]
    else:
        in_file = input("Type input .asm file name: ").strip()

    # 2) Compute output path: same directory as input file
    asm_dir = os.path.dirname(os.path.abspath(in_file))
    out_file = os.path.join(asm_dir, "instructions.mem")

    binary_instrs = []

    # 3) Read, assemble, collect instructions
    with open(in_file, "r") as f:
        for lineno, line in enumerate(f, 1):
            instr = assemble_line(line, lineno)
            if instr is not None:
                bits = f"{instr:025b}"   # 25-bit binary string
                binary_instrs.append(bits)

    # 4) Write instructions.mem in the same folder
    with open(out_file, "w") as f:
        for bits in binary_instrs:
            f.write(bits + "\n")

    print(f"Wrote {len(binary_instrs)} instructions.")
    print(f"Output in file: {out_file}")


if __name__ == "__main__":
    main()
```

Listing 1: Python assembler for the 25-bit SIMD instruction set

2 Part B: VHDL Source Code

This section lists the synthesizable VHDL RTL modules that implement the pipelined SIMD multimedia unit.

2.1 Top-Level Pipeline

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pipelined_simd_multimedia_unit is
  port(
    clk_if       : in  std_logic;
    clk_id       : in  std_logic;
    clk_ex       : in  std_logic;
    clk_wb       : in  std_logic;
    rst          : in  std_logic;

    rd_out       : out std_logic_vector(127 downto 0); -- ALU result (EX)
    instr_id_out : out std_logic_vector(24 downto 0);  -- instr in ID/EX
    pc_if_out    : out unsigned(5 downto 0);           -- PC in IF

    -- debug: WB control
    wr_en_dbg    : out std_logic;
    df_addr_dbg  : out std_logic_vector(4 downto 0)
  );
end entity;

architecture rtl of pipelined_simd_multimedia_unit is

  -- IF stage
  signal instr_if  : std_logic_vector(24 downto 0);
  signal pc_if     : unsigned(5 downto 0);

  -- ID stage
  signal instr_id  : std_logic_vector(24 downto 0);
  signal pc_id     : unsigned(5 downto 0);

  -- register file outputs
  signal rs1_rf, rs2_rf, rs3_rf : std_logic_vector(127 downto 0);

  -- EX stage regs (ID/EX outputs)
  signal instr_ex  : std_logic_vector(24 downto 0);
  signal rs1_ex, rs2_ex, rs3_ex : std_logic_vector(127 downto 0);

  -- forwarded operands into ALU
  signal rs1_fwd, rs2_fwd, rs3_fwd : std_logic_vector(127 downto 0);

  -- ALU output
  signal rd_alu : std_logic_vector(127 downto 0);

```

```
46    -- EX forwarding source **registered**
47    signal ex_addr_reg : std_logic_vector(4 downto 0);
48    signal ex_data_reg : std_logic_vector(127 downto 0);
49    signal ex_we_reg   : std_logic;
50
51    -- WB forwarding / regfile writeback source
52    signal wb_addr_sig : std_logic_vector(4 downto 0);
53    signal wb_data_sig : std_logic_vector(127 downto 0);
54    signal wb_we_sig   : std_logic;
55
56    -- same function as in exwb_reg
57    function will_write(i : std_logic_vector(24 downto 0)) return std_logic is
58    begin
59      if    i(24) = '0' then                    -- LI
60        return '1';
61      elsif i(24 downto 23) = "10" then         -- R4
62        return '1';
63      elsif i(24 downto 23) = "11" then         -- R3
64        if i(18 downto 15) = "0000" then        -- NOP
65          return '0';
66        else
67          return '1';
68        end if;
69      else
70        return '0';
71      end if;
72    end function;
73
74  begin
75    ------------------------------------------------------------------
76    -- IF stage: instruction buffer based on pc
77    ------------------------------------------------------------------
78    U_IF : entity work.instr_buffer
79      port map(
80        clk       => clk_if,
81        rst       => rst,
82        instr_out => instr_if,
83        pc_out    => pc_if
84      );
85
86    pc_if_out <= pc_if;
87
88    ------------------------------------------------------------------
89    -- IF/ID pipeline register
90    ------------------------------------------------------------------
91    U_IF_ID : entity work.reg_if_id
92      port map(
93        clk       => clk_id,
94        rst       => rst,
95        instr_in  => instr_if,
96        pc_in     => pc_if,
97        instr_out => instr_id,
98        pc_out    => pc_id
99      );
100
101   instr_id_out <= instr_id;
102
103   ------------------------------------------------------------------
104   -- Register file (3R/1W) in ID stage
```

```
105   ------------------------------------------------------------------
106   U_RF : entity work.registers
107     port map(
108       instructioni => instr_id,
109       instruction  => open,
110       clk          => clk_id,
111
112       wr_en        => wb_we_sig,
113       wr_addr      => wb_addr_sig,
114       wr_data      => wb_data_sig,
115
116       rs1          => rs1_rf,
117       rs2          => rs2_rf,
118       rs3          => rs3_rf
119     );
120
121   ------------------------------------------------------------------
122   -- ID/EX pipeline register
123   ------------------------------------------------------------------
124   U_ID_EX : entity work.reg_id_ex
125     port map(
126       instructioni => instr_id,
127       instruction  => instr_ex,
128       clk          => clk_ex,
129
130       rs1in        => rs1_rf,
131       rs2in        => rs2_rf,
132       rs3in        => rs3_rf,
133
134       rs1          => rs1_ex,
135       rs2          => rs2_ex,
136       rs3          => rs3_ex
137     );
138
139   ------------------------------------------------------------------
140   -- EX forwarding source REGISTER
141   --   These hold the result/rd of the instruction that just finished EX
142   --   and are used as the EX->ID/EX forwarding source *next* time.
143   ------------------------------------------------------------------
144   ex_reg_proc : process(clk_ex, rst)
145   begin
146     if rst = '1' then
147       ex_addr_reg <= (others => '0');
148       ex_data_reg <= (others => '0');
149       ex_we_reg   <= '0';
150     elsif rising_edge(clk_ex) then
151       ex_addr_reg <= instr_ex(4 downto 0);
152       ex_data_reg <= rd_alu;
153       ex_we_reg   <= will_write(instr_ex);
154     end if;
155   end process;
156
157   ------------------------------------------------------------------
158   -- Forwarding unit: EX(reg) + WB -> operands
159   ------------------------------------------------------------------
160   U_FWD : entity work.data_forwarding_unit
161     port map(
162       instr_id => instr_ex,
163       rs1i     => rs1_ex,
```

```
      rs2i     => rs2_ex,
      rs3i     => rs3_ex,

      ex_addr  => ex_addr_reg,
      ex_data  => ex_data_reg,
      ex_we    => ex_we_reg,

      wb_addr  => wb_addr_sig,
      wb_data  => wb_data_sig,
      wb_we    => wb_we_sig,

      rs1      => rs1_fwd,
      rs2      => rs2_fwd,
      rs3      => rs3_fwd
    );

  ----------------------------------------------------------------------
  -- ALU (pure combinational)
  ----------------------------------------------------------------------
  U_ALU : entity work.alu
    port map(
      instruction => instr_ex,
      rs1         => rs1_fwd,
      rs2         => rs2_fwd,
      rs3         => rs3_fwd,
      rd          => rd_alu
    );

  rd_out <= rd_alu;  -- observe EX result

  ----------------------------------------------------------------------
  -- EX/WB register, produces WB forwarding source + RF writeback
  ----------------------------------------------------------------------
  U_EXWB : entity work.exwb_reg
    port map(
      clk         => clk_wb,
      instruction => instr_ex,
      rd_data     => rd_alu,

      df_addr     => wb_addr_sig,
      df_data     => wb_data_sig,
      wr_en       => wb_we_sig
    );

  -- debug exports
  wr_en_dbg   <= wb_we_sig;
  df_addr_dbg <= wb_addr_sig;

end architecture;
```

Listing 2: Top-level pipelined SIMD multimedia unit

2.2 Instruction Buffer

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```
4
5  library std;
6  use std.textio.all;
7
8  entity instr_buffer is
9    port(
10     clk       : in  std_logic;
11     rst       : in  std_logic;
12     instr_out : out std_logic_vector(24 downto 0); -- current instruction
13     pc_out    : out unsigned(5 downto 0)           -- current PC (0..63)
14   );
15 end entity instr_buffer;
16
17 architecture behavioral of instr_buffer is
18
19   -- 64 x 25-bit memory
20   type mem_t is array (0 to 63) of std_logic_vector(24 downto 0);
21   signal mem : mem_t := (others => (others => '0'));
22
23   -- program counter
24   signal pc  : unsigned(5 downto 0) := (others => '0');
25
26   -- helper: convert bit_vector -> std_logic_vector
27   function bv_to_slv(b : bit_vector) return std_logic_vector is
28     variable res : std_logic_vector(b'range);
29   begin
30     for i in b'range loop
31       if b(i) = '0' then
32         res(i) := '0';
33       else
34         res(i) := '1';
35       end if;
36     end loop;
37     return res;
38   end function;
39
40 begin
41
42   ----------------------------------------------------------------------------
43   -- ONE-TIME LOAD FROM "instructions.txt"
44   --  Each line must be exactly 25 chars of 0/1, no spaces.
45   ----------------------------------------------------------------------------
46   file_load_proc : process
47     file f  : text open read_mode is
48       "instructions.mem";
49     variable L  : line;
50     variable bv : bit_vector(24 downto 0);
51     variable i  : integer := 0;
52   begin
53     -- default clear
54     for idx in 0 to 63 loop
55       mem(idx) <= (others => '0');
56     end loop;
57
58     i := 0;
59     while (not endfile(f)) and (i <= 63) loop
60       readline(f, L);
61       -- read 25 bits "000...1" into bit_vector
62       read(L, bv);
```

```
63        mem(i) <= bv_to_slv(bv);
64        report "instr_buffer: loaded instruction " & integer'image(i);
65        i := i + 1;
66      end loop;
67
68      report "instr_buffer: finished loading "
69             & integer'image(i) & " line(s) from instructions.txt";
70
71      wait;  -- run once
72    end process;
73
74    -------------------------------------------------------------------------
75    -- MAIN: on each clk, output mem(pc) and increment pc
76    -------------------------------------------------------------------------
77    main_proc : process(clk, rst)
78    begin
79      if rst = '1' then
80        pc        <= (others => '0');
81        instr_out <= (others => '0');
82        pc_out    <= (others => '0');
83
84      elsif rising_edge(clk) then
85        instr_out <= mem(to_integer(pc));
86        pc_out    <= pc;
87
88        if pc = "111111" then
89          pc <= (others => '0');
90        else
91          pc <= pc + 1;
92        end if;
93      end if;
94    end process;
95
96  end architecture behavioral;
```

Listing 3: Instruction buffer and program counter

2.3 Register File

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity registers is
6    port(
7      instructioni : in  std_logic_vector(24 downto 0);
8      instruction  : out std_logic_vector(24 downto 0);
9      clk          : in  std_logic;
10
11     wr_en        : in  std_logic;
12     wr_addr      : in  std_logic_vector(4 downto 0);
13     wr_data      : in  std_logic_vector(127 downto 0);
14
15     rs1          : out std_logic_vector(127 downto 0);
16     rs2          : out std_logic_vector(127 downto 0);
17     rs3          : out std_logic_vector(127 downto 0)
18   );
```

```
19 end entity;
20
21 architecture memory of registers is
22   subtype idx_t is integer range 0 to 31;
23   type mem_t is array (idx_t) of std_logic_vector(127 downto 0);
24   signal mem : mem_t := (others => (others => '0'));
25
26   -- Constrain to 5 bits to avoid COMP96_0149
27   function to_idx(a : std_logic_vector(4 downto 0)) return idx_t is
28   begin
29     return idx_t'(to_integer(unsigned(a)));
30   end function;
31
32   -- Helper: read one word from mem
33   function fetch_data(addr : std_logic_vector(4 downto 0); m : mem_t)
34     return std_logic_vector is
35   begin
36     return m(to_idx(addr));
37   end function;
38 begin
39
40   ----------------------------------------------------------------
41   -- WRITE PORT: synchronous 1W
42   ----------------------------------------------------------------
43   write_proc : process(clk)
44   begin
45     if rising_edge(clk) then
46       if wr_en = '1' then
47         mem(to_idx(wr_addr)) <= wr_data;
48       end if;
49
50       -- pass-through instruction (latched if you want it clocked)
51       instruction <= instructioni;
52     end if;
53   end process;
54
55   ----------------------------------------------------------------
56   -- READ PORTS: combinational 3R based on current instructioni
57   ----------------------------------------------------------------
58   read_proc : process(instructioni, mem)
59   begin
60     -- defaults
61     rs1 <= (others => '0');
62     rs2 <= (others => '0');
63     rs3 <= (others => '0');
64
65     if instructioni(24) = '0' then                    -- LI
66       -- LI uses rd as "source" when building new value
67       rs1 <= fetch_data(instructioni(4 downto 0), mem);
68
69     elsif instructioni(24 downto 23) = "10" then      -- R4
70       rs1 <= fetch_data(instructioni(9  downto 5),  mem);
71       rs2 <= fetch_data(instructioni(14 downto 10), mem);
72       rs3 <= fetch_data(instructioni(19 downto 15), mem);
73
74     elsif instructioni(24 downto 23) = "11" then      -- R3
75       rs1 <= fetch_data(instructioni(9  downto 5),  mem);
76       rs2 <= fetch_data(instructioni(14 downto 10), mem);
77       -- rs3 unused for R3
```

```
78
79      else
80        -- illegal / unused opcode: keep zeros
81        null;
82      end if;
83    end process;
84
85  end architecture memory;
```

Listing 4: 32×128-bit register file

2.4 IF/ID and ID/EX Pipeline Registers

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity reg_if_id is
6    port(
7      clk       : in  std_logic;
8      rst       : in  std_logic;
9
10     -- from IF stage (buffer)
11     instr_in  : in  std_logic_vector(24 downto 0);
12     pc_in     : in  unsigned(5 downto 0);
13
14     -- to ID stage
15     instr_out : out std_logic_vector(24 downto 0);
16     pc_out    : out unsigned(5 downto 0)
17   );
18 end entity;
19
20 architecture rtl of reg_if_id is
21 begin
22   process(clk, rst)
23   begin
24     if rst = '1' then
25       instr_out <= (others => '0');
26       pc_out    <= (others => '0');
27     elsif rising_edge(clk) then
28       instr_out <= instr_in;
29       pc_out    <= pc_in;
30     end if;
31   end process;
32 end architecture;
```

Listing 5: IF/ID pipeline register

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity reg_id_ex is
6    port(
7      instructioni  : in  std_logic_vector(24 downto 0);
8      instruction   : out std_logic_vector(24 downto 0);
9      clk           : in  std_logic;
```

```
10
11      rs1in          : in  std_logic_vector(127 downto 0);
12      rs2in          : in  std_logic_vector(127 downto 0);
13      rs3in          : in  std_logic_vector(127 downto 0);
14
15      rs1            : out std_logic_vector(127 downto 0);
16      rs2            : out std_logic_vector(127 downto 0);
17      rs3            : out std_logic_vector(127 downto 0)
18    );
19  end entity;
20
21  architecture buff of reg_id_ex is
22  begin
23    process(clk)
24    begin
25      if rising_edge(clk) then
26        -- latch operands into EX
27        rs1        <= rs1in;
28        rs2        <= rs2in;
29        rs3        <= rs3in;
30
31        -- ?? latch instruction for EX stage
32        instruction <= instructioni;
33      end if;
34    end process;
35  end architecture;
```

Listing 6: ID/EX pipeline register

2.5 Data Forwarding Unit

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.numeric_std.all;
4
5   entity data_forwarding_unit is
6     port(
7       instr_id  : in  std_logic_vector(24 downto 0); -- ID/EX instr
8       rs1i, rs2i, rs3i : in std_logic_vector(127 downto 0);
9
10      -- EX source
11      ex_addr   : in  std_logic_vector(4 downto 0);
12      ex_data   : in  std_logic_vector(127 downto 0);
13      ex_we     : in  std_logic;
14
15      -- WB source
16      wb_addr   : in  std_logic_vector(4 downto 0);
17      wb_data   : in  std_logic_vector(127 downto 0);
18      wb_we     : in  std_logic;
19
20      rs1, rs2, rs3 : out std_logic_vector(127 downto 0)
21    );
22  end entity;
23
24  architecture rtl of data_forwarding_unit is
25  begin
26    process(all)
```

```
  begin
    rs1 <= rs1i;
    rs2 <= rs2i;
    rs3 <= rs3i;

    if instr_id(24) = '0' then             -- LI
      -- rs1 = rd(4:0)
      if ex_we='1' and instr_id(4 downto 0)=ex_addr then
        rs1 <= ex_data;
      elsif wb_we='1' and instr_id(4 downto 0)=wb_addr then
        rs1 <= wb_data;
      end if;

    elsif instr_id(24 downto 23) = "11" then  -- R3
      -- rs1 = 9:5, rs2 = 14:10
      if ex_we='1' and instr_id(9 downto 5)=ex_addr then
        rs1 <= ex_data;
      elsif wb_we='1' and instr_id(9 downto 5)=wb_addr then
        rs1 <= wb_data;
      end if;

      if ex_we='1' and instr_id(14 downto 10)=ex_addr then
        rs2 <= ex_data;
      elsif wb_we='1' and instr_id(14 downto 10)=wb_addr then
        rs2 <= wb_data;
      end if;

      -- (R4 case similar, adding rs3 checks)
    end if;
  end process;
end architecture;
```

Listing 7: Data forwarding unit

2.6 SIMD ALU

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

--=============================================================================
--  ALU (128-bit SIMD): LI / R4 MAC / R3 vector ops
--  instruction[24]    : 0 = LI, 1 = R3/R4
--  instruction[24:23] : "10" = R4  (MAC family, see table)
--                       "11" = R3  (packed ops, see table)
--=============================================================================

entity alu is
    port (
        instruction  : in  std_logic_vector(24 downto 0);
        rs1, rs2, rs3: in  std_logic_vector(127 downto 0);
        rd           : out std_logic_vector(127 downto 0)
    );
end entity;

architecture behavioral of alu is

```

```
22 --================================================================================
23 --  CONSTANTS / SIZES
24 --================================================================================
25 constant TOT_BITS    : integer := 128;
26
27 -- -- LI (load-immediate) shape: 8x16
28 constant li_LANES    : integer := 8;
29 constant li_LANE_W   : integer := 16;
30
31 -- -- R4 (MAC) shape: 2x64
32 constant r4_LANES    : integer := 2;
33 constant r4_LANE_W   : integer := 64;
34
35 -- -- R3 (packed ops) shape: 4x32 words, or 8x16 halfwords
36 constant r3_LANES    : integer := 4;
37 constant r3_LANE_W   : integer := 32;
38 constant WIDTH_BITS  : integer := r3_LANES * r3_LANE_W;  -- 128
39 constant HALF_W      : integer := 16;                   -- 16
40 constant NUM_HALVES  : integer := WIDTH_BITS / HALF_W;  -- 8
41 constant NUM_WORDS   : integer := r3_LANES;             -- 4
42
43 --================================================================================
44 --  R4: Saturation helpers (per spec)
45 --================================================================================
46
47 -- 32-bit saturate of a 64-bit accumulator
48 function sat32(x : signed(63 downto 0)) return signed is
49   constant MAX32 : signed(31 downto 0) := signed(x"7FFFFFFF");
50   constant MIN32 : signed(31 downto 0) := signed(x"80000000");
51 begin
52   if    x > resize(MAX32, x'length) then return MAX32;
53   elsif x < resize(MIN32, x'length) then return MIN32;
54   else                                   return resize(x, 32);
55   end if;
56 end function;
57
58 -- 64-bit saturate of a 128-bit accumulator
59 function sat64(x : signed(127 downto 0)) return signed is
60   constant MAX64 : signed(63 downto 0) := signed(x"7FFFFFFFFFFFFFFF");
61   constant MIN64 : signed(63 downto 0) := signed(x"8000000000000000");
62 begin
63   if    x > resize(MAX64, x'length) then return MAX64;
64   elsif x < resize(MIN64, x'length) then return MIN64;
65   else                                   return resize(x, 64);
66   end if;
67 end function;
68
69 --================================================================================
70 --  R3: Opcode nibble (instruction[18:15]) definitions
71 --  (Matches your table: NOP, SHRHI, AU, CNT1H, AHS, OR, BCW, MAXWS, MINWS,
72 --   MLHU, MLHCU, AND, CLZW, ROTW, SFWU, SFHS)
73 --================================================================================
74 constant OPC_NOP   : std_logic_vector(3 downto 0) := "0000";
75 constant OPC_SHRHI : std_logic_vector(3 downto 0) := "0001";
76 constant OPC_AU    : std_logic_vector(3 downto 0) := "0010";
77 constant OPC_CNT1H : std_logic_vector(3 downto 0) := "0011";
78 constant OPC_AHS   : std_logic_vector(3 downto 0) := "0100";
79 constant OPC_OR    : std_logic_vector(3 downto 0) := "0101";
80 constant OPC_BCW   : std_logic_vector(3 downto 0) := "0110";
```

```
constant OPC_MAXWS : std_logic_vector(3 downto 0) := "0111";
constant OPC_MINWS : std_logic_vector(3 downto 0) := "1000";
constant OPC_MLHU  : std_logic_vector(3 downto 0) := "1001";
constant OPC_MLHCU : std_logic_vector(3 downto 0) := "1010";
constant OPC_AND   : std_logic_vector(3 downto 0) := "1011";
constant OPC_CLZW  : std_logic_vector(3 downto 0) := "1100";
constant OPC_ROTW  : std_logic_vector(3 downto 0) := "1101";
constant OPC_SFWU  : std_logic_vector(3 downto 0) := "1110";
constant OPC_SFHS  : std_logic_vector(3 downto 0) := "1111";

--================================================================================
--  R3: Halfword saturation helpers
--================================================================================
constant S16_MAX : signed(15 downto 0) := to_signed( 32767, 16); -- 0x7FFF
constant S16_MIN : signed(15 downto 0) := to_signed(-32768, 16); -- 0x8000

function clamp16(x17 : signed(16 downto 0)) return signed is
begin
  if    x17 > to_signed( 32767, 17) then return S16_MAX;
  elsif x17 < to_signed(-32768, 17) then return S16_MIN;
  else                                   return signed(resize(x17, 16));
  end if;
end function;

function sat16(x : signed(15 downto 0)) return signed is
begin
  if    x > S16_MAX then return S16_MAX;
  elsif x < S16_MIN then return S16_MIN;
  else                   return x;
  end if;
end function;

begin
process(all)

  --================================================================================
  --  LI (instruction[24] = '0'): load 16-bit immediate into one of 8 lanes
  --  Lane index = instruction[23:21], imm16 = instruction[20:5]
  --================================================================================
  variable li_tmp      : std_logic_vector(TOT_BITS-1 downto 0);
  variable li_lane_idx : integer range 0 to li_LANES-1;
  variable li_imm16    : std_logic_vector(li_LANE_W-1 downto 0);

  --================================================================================
  --  R4 (instruction[24:23] = "10"): MAC family (see your R4 table)
  --    bit22=longInt (0=int halves, 1=long 64)
  --    bit21=highLow (0=low, 1=high half selection)
  --    bit20=subAdd  (0=add, 1=sub)
  --================================================================================
  variable longInt, subAdd, highLow : std_logic;
  variable outv    : std_logic_vector(r4_LANES*r4_LANE_W-1 downto 0);

  -- per-lane 64-bit signed views
  variable x1_64, x2_64, x3_64 : signed(r4_LANE_W-1 downto 0);

  -- INT mode temps (2x32 halves inside each 64)
  variable dst_64              : signed(63 downto 0);
  variable a16, b16            : signed(15 downto 0);
  variable prod32              : signed(31 downto 0);
```

```
140   variable acc64               : signed(63 downto 0);
141
142   -- LONG mode temps (select 32-bit half, 32x32->64, accumulate into 64)
143   variable a32, b32            : signed(31 downto 0);
144   variable prod64              : signed(63 downto 0);
145   variable acc128              : signed(127 downto 0);
146   variable res64               : signed(63 downto 0);
147
148   variable lo32, hi32          : integer;
149
150   --============================================================================
151   --  R3 (instruction[24:23] = "11"): packed ops case on instruction[18:15]
152   --============================================================================
153   variable r3_tmp : std_logic_vector(WIDTH_BITS-1 downto 0);
154   variable opc4   : std_logic_vector(3 downto 0);
155
156   variable sh4  : integer  range 0 to 15;         -- rs2[13:10]
157   variable imm5 : unsigned(4 downto 0);           -- rs2[14:10]
158
159   -- temps
160   variable r3_a16, r3_b16, r3_r16 : signed(15 downto 0);
161   variable aw, bw, rw             : signed(31 downto 0);
162   variable u16, v16               : unsigned(15 downto 0);
163   variable u32, v32, r32          : unsigned(31 downto 0);
164   variable r3_prod32              : unsigned(31 downto 0);
165   variable cnt                    : integer range 0 to 32;
166   variable k                      : integer range 0 to 31;
167   variable word_slv               : std_logic_vector(31 downto 0);
168
169 begin
170   if instruction(24) = '0' then
171     --=== LI ================================================================
172     li_tmp := rs1;
173     li_lane_idx := to_integer(unsigned(instruction(23 downto 21)));
174     li_imm16    := instruction(20 downto 5);
175
176     if (li_lane_idx >= 0) and (li_lane_idx < li_LANES) then
177       li_tmp((li_lane_idx+1)*li_LANE_W-1 downto li_lane_idx*li_LANE_W) := li_imm16;
178     end if;
179
180     rd <= li_tmp;
181
182   elsif instruction(24 downto 23) = "10" then
183     --=== R4: MAC family ====================================================
184     outv    := (others => '0');
185     longInt := instruction(22);
186   subAdd  := instruction(21);
187     highLow := instruction(20);
188
189
190     for lane in 0 to r4_LANES-1 loop
191       x1_64 := signed(rs1((lane+1)*64-1 downto lane*64));
192       x2_64 := signed(rs2((lane+1)*64-1 downto lane*64));
193       x3_64 := signed(rs3((lane+1)*64-1 downto lane*64));
194
195       if longInt = '0' then
196         -- -- INT (000/001/010/011): 16x16->32 add/sub into each 32-bit half with
              saturation
197         dst_64 := x1_64;
```

```
198
199         for half in 0 to 1 loop
200           lo32 := half*32;
201           hi32 := lo32+31;
202
203           if highLow = '0' then
204             -- Low 16 of each 32 field (000 / 010)
205             a16 := signed(x2_64(lo32+15 downto lo32));
206             b16 := signed(x3_64(lo32+15 downto lo32));
207           else
208             -- High 16 of each 32 field (001 / 011)
209             a16 := signed(x2_64(hi32 downto hi32-15));
210             b16 := signed(x3_64(hi32 downto hi32-15));
211           end if;
212
213           prod32 := signed(a16) * signed(b16);
214
215           if subAdd = '0' then
216             -- Add with saturation (000/001)
217             acc64 := resize(signed(dst_64(hi32 downto lo32)), 64) +
                    resize(signed(prod32), 64);
218           else
219             -- Sub with saturation (010/011)
220             acc64 := resize(signed(dst_64(hi32 downto lo32)), 64) -
                    resize(signed(prod32), 64);
221           end if;
222
223           dst_64(hi32 downto lo32) := sat32(acc64);  -- clamp to 32-bit
224         end loop;
225
226         outv((lane+1)*64-1 downto lane*64) := std_logic_vector(dst_64);
227
228       else
229         -- -- LONG (100/101/110/111): pick 32 half, 32x32->64, add/sub into 64
              with saturation
230         if highLow = '0' then
231           -- Low 32 (100 / 110)
232           a32 := signed(x2_64(31 downto 0));
233           b32 := signed(x3_64(31 downto 0));
234         else
235           -- High 32 (101 / 111)
236           a32 := signed(x2_64(63 downto 32));
237           b32 := signed(x3_64(63 downto 32));
238         end if;
239
240         prod64 := signed(a32) * signed(b32);
241
242         if subAdd = '0' then
243           -- Add (100/101)
244           acc128 := resize(signed(x1_64), 128) + resize(signed(prod64), 128);
245         else
246           -- Sub (110/111)
247           acc128 := resize(signed(x1_64), 128) - resize(signed(prod64), 128);
248         end if;
249
250         res64 := sat64(acc128);  -- clamp to 64-bit
251         outv((lane+1)*64-1 downto lane*64) := std_logic_vector(res64);
252       end if;
253     end loop;
```

```
254
255     rd <= outv;
256
257   elsif instruction(24 downto 23) = "11" then
258     --=== R3: packed ops ========================================================
259     r3_tmp := (others => '0');
260     opc4   := instruction(18 downto 15);
261     sh4    := to_integer(unsigned(instruction(13 downto 10)));
262     imm5   := unsigned(instruction(14 downto 10));
263
264     case opc4 is
265
266       -- xxxx0000 : NOP (do nothing)
267       when OPC_NOP =>
268         null;
269
270       -- xxxx0001 : SHRHI  (packed 16-bit logical right shift by sh4)
271       when OPC_SHRHI =>
272         for i in 0 to NUM_HALVES-1 loop
273           r3_tmp(i*HALF_W+15 downto i*HALF_W) :=
274             std_logic_vector(shift_right(unsigned(rs1(i*HALF_W+15 downto
                  i*HALF_W)), sh4));
275         end loop;
276
277       -- xxxx0010 : AU     (packed 32-bit unsigned add)
278       when OPC_AU =>
279         for w in 0 to NUM_WORDS-1 loop
280           r3_tmp(w*32+31 downto w*32) :=
281             std_logic_vector(unsigned(rs1(w*32+31 downto w*32)) +
282                              unsigned(rs2(w*32+31 downto w*32)));
283         end loop;
284
285       -- xxxx0011 : CNT1H  (count ones in each 16-bit halfword)
286       when OPC_CNT1H =>
287         for i in 0 to NUM_HALVES-1 loop
288           cnt := 0;
289           for b in 0 to 15 loop
290             if rs1(i*HALF_W + b) = '1' then cnt := cnt + 1; end if;
291           end loop;
292           r3_tmp(i*HALF_W+15 downto i*HALF_W) := std_logic_vector(to_unsigned(cnt,
                16));
293         end loop;
294
295       -- xxxx0100 : AHS    (packed 16-bit signed add with saturation)
296       when OPC_AHS =>
297         for i in 0 to NUM_HALVES-1 loop
298           r3_a16 := signed(rs1(i*16+15 downto i*16));
299           r3_b16 := signed(rs2(i*16+15 downto i*16));
300           r3_r16 := clamp16(resize(r3_a16,17) + resize(r3_b16,17));
301           r3_tmp(i*16+15 downto i*16) := std_logic_vector(r3_r16);
302         end loop;
303
304       -- xxxx0101 : OR     (bitwise OR)
305       when OPC_OR =>
306         r3_tmp := rs1 or rs2;
307
308       -- xxxx0110 : BCW    (broadcast word: rs1[127:96] into all four 32 slots)
309       when OPC_BCW =>
310         for w in 0 to NUM_WORDS-1 loop
```

```
311             r3_tmp(w*32+31 downto w*32) := rs1(127 downto 96);
312           end loop;
313
314         -- xxxx0111 : MAXWS  (per-word signed max)
315         when OPC_MAXWS =>
316           for w in 0 to NUM_WORDS-1 loop
317             aw := signed(rs1(w*32+31 downto w*32));
318             bw := signed(rs2(w*32+31 downto w*32));
319             if aw >= bw then rw := aw; else rw := bw; end if;
320             r3_tmp(w*32+31 downto w*32) := std_logic_vector(rw);
321           end loop;
322
323         -- xxxx1000 : MINWS  (per-word signed min)
324         when OPC_MINWS =>
325           for w in 0 to NUM_WORDS-1 loop
326             aw := signed(rs1(w*32+31 downto w*32));
327             bw := signed(rs2(w*32+31 downto w*32));
328             if aw <= bw then rw := aw; else rw := bw; end if;
329             r3_tmp(w*32+31 downto w*32) := std_logic_vector(rw);
330           end loop;
331
332         -- xxxx1001 : MLHU   (low16*low16 -> 32, unsigned)
333         when OPC_MLHU =>
334           for w in 0 to NUM_WORDS-1 loop
335             u16        := unsigned(rs1(w*32+15 downto w*32));
336             v16        := unsigned(rs2(w*32+15 downto w*32));
337             r3_prod32  := u16 * v16;  -- 16x16 -> 32
338             r3_tmp(w*32+31 downto w*32) := std_logic_vector(r3_prod32);
339           end loop;
340
341         -- xxxx1010 : MLHCU  (low16 * imm5 -> 32, unsigned)
342         when OPC_MLHCU =>
343           for w in 0 to NUM_WORDS-1 loop
344             u16        := unsigned(rs1(w*32+15 downto w*32));
345             r3_prod32  := u16 * resize(imm5, 16);
346             r3_tmp(w*32+31 downto w*32) := std_logic_vector(r3_prod32);
347           end loop;
348
349         -- xxxx1011 : AND    (bitwise AND)
350         when OPC_AND =>
351           r3_tmp := rs1 and rs2;
352
353         -- xxxx1100 : CLZW   (count leading zeros per 32-bit word)
354         when OPC_CLZW =>
355           for w in 0 to NUM_WORDS-1 loop
356             word_slv := rs1(w*32+31 downto w*32);
357             cnt := 0;
358             for b in 31 downto 0 loop
359               if word_slv(b) = '0' then cnt := cnt + 1; else exit; end if;
360             end loop;
361             r3_tmp(w*32+31 downto w*32) := std_logic_vector(to_unsigned(cnt, 32));
362           end loop;
363
364         -- xxxx1101 : ROTW   (rotate-right per 32-bit word by rs2[4:0])
365         when OPC_ROTW =>
366           for w in 0 to NUM_WORDS-1 loop
367             u32 := unsigned(rs1(w*32+31 downto w*32));
368             v32 := unsigned(rs2(w*32+31 downto w*32));
369             k   := to_integer(v32(4 downto 0)) mod 32;
```

```
370           if k = 0 then
371             r32 := u32;
372           else
373             r32 := shift_right(u32, k) or shift_left(u32, 32 - k);
374           end if;
375           r3_tmp(w*32+31 downto w*32) := std_logic_vector(r32);
376         end loop;
377
378       -- xxxx1110 : SFWU   (per-word unsigned subtract: rs2 - rs1)
379       when OPC_SFWU =>
380         for w in 0 to NUM_WORDS-1 loop
381           r3_tmp(w*32+31 downto w*32) :=
382             std_logic_vector(unsigned(rs2(w*32+31 downto w*32)) -
383                              unsigned(rs1(w*32+31 downto w*32)));
384         end loop;
385
386       -- xxxx1111 : SFHS   (per-halfword signed subtract with saturation: rs2 -
              rs1)
387       when OPC_SFHS =>
388         for i in 0 to NUM_HALVES-1 loop
389           r3_a16 := signed(rs1(i*16+15 downto i*16));
390           r3_b16 := signed(rs2(i*16+15 downto i*16));
391           r3_r16 := clamp16(resize(r3_b16,17) - resize(r3_a16,17));
392           r3_tmp(i*16+15 downto i*16) := std_logic_vector(r3_r16);
393         end loop;
394
395       when others =>
396         null;
397
398     end case;
399
400     rd <= r3_tmp;
401
402   end if;
403 end process;
404 end architecture;
```

Listing 8: 128-bit SIMD ALU

2.7 Writeback Stage

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  -- EX/WB register / writeback metadata for forwarding
6  entity exwb_reg is
7    port(
8      clk          : in  std_logic;
9      instruction  : in  std_logic_vector(24 downto 0);  -- current instr in EX stage
10     rd_data      : in  std_logic_vector(127 downto 0); -- ALU result from EX
11
12     -- To forwarding unit + register file writeback
13     df_addr      : out std_logic_vector(4 downto 0);   -- rd
14     df_data      : out std_logic_vector(127 downto 0); -- value to forward/write
15     wr_en        : out std_logic                       -- valid write this instr?
16   );
```

```
end entity;

architecture behavioral of exwb_reg is
  -- returns '1' if this instruction writes rd, else '0'
  function will_write(i : std_logic_vector(24 downto 0)) return std_logic is
  begin
    if    i(24) = '0' then                  -- LI
      return '1';
    elsif i(24 downto 23) = "10" then       -- R4
      return '1';
    elsif i(24 downto 23) = "11" then       -- R3
      -- NOP is opcode 0000 at [18:15]; others write
      if i(18 downto 15) = "0000" then
        return '0';
      else
        return '1';
      end if;
    else                                     -- illegal / reserved -> no write
      return '0';
    end if;
  end function;
begin
  process(clk)
  begin
    if rising_edge(clk) then
      df_addr <= instruction(4 downto 0);        -- rd field
      df_data <= rd_data;                        -- EX result
      wr_en   <= will_write(instruction);        -- write valid?
    end if;
  end process;
end architecture;
```

Listing 9: Writeback stage and EX/WB metadata

3 Part C: VHDL Testbenches

The following listings contain the self-checking testbenches used to verify each RTL block and the integrated pipeline.

3.1 ALU Self-Checking Testbench

```
-- ============================================================
--  ALU SIMD Testbench (2 lanes   64-bit = 128-bit datapath)
--  This TB exercises:
--    - LI (load-immediate to 16b lane halves)
--    - R4 family (MAC-style INT/LONG ops; forms 000..111)
--    - R3 family (packed ops; opcodes 0000..1111)
--  All checks are self-verifying with assertions.
-- ============================================================

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu_sc_tb is
end entity;
```

```
16
17 architecture sim of alu_sc_tb is
18
19   -------------------------------------------------------------
20   -- Constant configuration of the datapath under test
21   -------------------------------------------------------------
22   constant LANE_W : integer := 64;  -- width of a lane (bits)
23   constant LANES  : integer := 2;   -- number of lanes (total datapath = 128 bits)
24
25   -------------------------------------------------------------
26   -- DUT ports (driven/observed by the testbench)
27   -------------------------------------------------------------
28   signal instruction : std_logic_vector(24 downto 0);                -- 25-bit
         instruction
29   signal rs1, rs2, rs3 : std_logic_vector(LANE_W*LANES-1 downto 0);  -- 128-bit
         source regs
30   signal rd : std_logic_vector(LANE_W*LANES-1 downto 0);             -- 128-bit
         dest reg
31
32   -------------------------------------------------------------
33   -- === HELPERS ===
34   -- Helpers build specific instruction encodings as 25-bit vectors.
35   -- Bit layout (common):
36   --   [24:23]  : major class  (00=LI, 10=R4, 11=R3)
37   --   [22:..]  : class-specific payload
38   -------------------------------------------------------------
39
40   -------------------------------------------------------------
41   -- LOAD IMMEDIATE (LI) instruction builder
42   -- Format (this TB s convention):
43   --   [24]     = 0                   ? identify LI class
44   --   [23:21]  = lane index (0..7)   ? which 64b lane to target
45   --   [20:5]   = imm16               ? 16-bit immediate written into that lane
46   --   [4:0]    = 0
47   --
48   -- Result (per your DUT): the 16-bit imm is injected into the selected lane
         position
49   -- and accumulated (rs1 <= rd) across several LI writes to assemble the 128-bit
         constant.
50   -------------------------------------------------------------
51   function mk_li(lane : integer; imm16 : std_logic_vector(15 downto 0))
52     return std_logic_vector is
53     variable t : std_logic_vector(24 downto 0) := (others => '0');
54   begin
55     t(24)            := '0';                                -- class = LI
56     t(23 downto 21)  := std_logic_vector(to_unsigned(lane, 3));
57     t(20 downto 5)   := imm16;                              -- payload
58     t(4 downto 0)    := (others => '0');
59     return t;
60   end function;
61
62   -------------------------------------------------------------
63   -- R4 INSTRUCTIONS (MAC family) builder
64   -- Format:
65   --   [24:23] = "10"               ? class = R4
66   --   [22:20] = form[2:0]          ? longInt / subAdd / highLow (after your fix)
67   --   [19:0]  = 0
68   -------------------------------------------------------------
69   function mk_r4(form : std_logic_vector(2 downto 0))
```

```
70      return std_logic_vector is
71      variable t : std_logic_vector(24 downto 0) := (others => '0');
72    begin
73      t(24 downto 23) := "10";           -- class = R4
74      t(22 downto 20) := form;           -- form bits (000..111)
75      t(19 downto 0)  := (others => '0');
76      return t;
77    end function;
78
79    ------------------------------------------------------------------
80    -- R3 INSTRUCTIONS (packed ops) builder
81    -- Format (matches your decoder):
82    --   [24:23] = "11"               ? class = R3
83    --   [22:19] = 0000               ? reserved/unused in this TB
84    --   [18:15] = opcode[3:0]        ? packed-op code (0000..1111)
85    --   [14:10] = shift/imm5         ? opcode-specific (e.g., SHRHI/MLHCU/ROTW)
86    --   [9:0]   = 0
87    ------------------------------------------------------------------
88    function mk_r3(code : std_logic_vector(3 downto 0); shift : std_logic_vector(4
          downto 0))
89      return std_logic_vector is
90      variable t : std_logic_vector(24 downto 0) := (others => '0');
91    begin
92      t(24 downto 23) := "11";           -- class = R3
93      t(22 downto 19) := (others => '0');-- reserved (kept zero for clarity)
94      t(18 downto 15) := code;           -- opcode
95      t(14 downto 10) := shift;          -- 5-bit field (imm/shift per opcode)
96      return t;
97    end function;
98
99  begin
100   ------------------------------------------------------------------
101   -- Device Under Test
102   ------------------------------------------------------------------
103   dut : entity work.alu
104     port map(
105       instruction => instruction,
106       rs1 => rs1, rs2 => rs2, rs3 => rs3,
107       rd => rd
108     );
109
110   ------------------------------------------------------------------
111   -- Stimulus process: drives instructions and operands,
112   -- checks results with a golden expected vector (exp).
113   ------------------------------------------------------------------
114   stim : process
115     variable exp : std_logic_vector(LANE_W*LANES-1 downto 0);
116   begin
117
118     ------------------------------------------------------------------
119     -- == LOAD IMMEDIATE ==
120     -- Build a 128-bit constant by writing one 16-bit lane piece at a time.
121     -- After each LI write, we assert rd and then copy rd back to rs1, so
122     -- we accumulate the full constant across the lanes.
123     --
124     -- Final target constant (top..bottom):
125     --   L7:L6:L5:L4:L3:L2:L1:L0 = DEAD FEED 8000 8135 6969 ABAD CAB0 AEFC
126     ------------------------------------------------------------------
127     rs1 <= (others => '0');
```

```
128     rs2 <= (others => '0');
129     rs3 <= (others => '0');
130
131     -- LANE 0 (lowest 16 bits of the 128-bit word)
132     instruction <= mk_li(0, x"AEFC");  wait for 1 ns;
133     exp := x"0000000000000000000000000000AEFC";
134     assert rd = exp report "LI in Lane 0 mismatched..." severity error;
135     rs1 <= rd;  wait for 1 ns;  -- accumulate
136
137     -- LANE 1
138     instruction <= mk_li(1, x"CAB1");  wait for 1 ns;
139     exp := x"00000000000000000000000CAB1AEFC";
140     assert rd = exp report "LI in Lane 1 mismatched..." severity error;
141     rs1 <= rd;  wait for 1 ns;
142
143     -- LANE 2
144     instruction <= mk_li(2, x"ABAD");  wait for 1 ns;
145     exp := x"000000000000000000ABADCAB1AEFC";
146     assert rd = exp report "LI in Lane 2 mismatched..." severity error;
147     rs1 <= rd;  wait for 1 ns;
148
149     -- LANE 3
150     instruction <= mk_li(3, x"6969");  wait for 1 ns;
151     exp := x"0000000000000006969ABADCAB1AEFC";
152     assert rd = exp report "LI in Lane 3 mismatched..." severity error;
153     rs1 <= rd;  wait for 1 ns;
154
155     -- LANE 4
156     instruction <= mk_li(4, x"8135");  wait for 1 ns;
157     exp := x"0000000000081356969ABADCAB1AEFC";
158     assert rd = exp report "LI in Lane 4 mismatched..." severity error;
159     rs1 <= rd;  wait for 1 ns;
160
161     -- LANE 5
162     instruction <= mk_li(5, x"8000");  wait for 1 ns;
163     exp := x"0000000800081356969ABADCAB1AEFC";
164     assert rd = exp report "LI in Lane 5 mismatched..." severity error;
165     rs1 <= rd;  wait for 1 ns;
166
167     -- LANE 6
168     instruction <= mk_li(6, x"FEED");  wait for 1 ns;
169     exp := x"0000FEED800081356969ABADCAB1AEFC";
170     assert rd = exp report "LI in Lane 6 mismatched..." severity error;
171     rs1 <= rd;  wait for 1 ns;
172
173     -- LANE 7 (top-most 16b)
174     instruction <= mk_li(7, x"DEAD");  wait for 1 ns;
175     exp := x"DEADFEED800081356969ABADCAB1AEFC";
176     assert rd = exp report "LI in Lane 7 mismatched..." severity error;
177     rs1 <= rd;  wait for 1 ns;
178
179     -- Overwrite lane 1 (prove we can update any slot)
180     instruction <= mk_li(1, x"CAB0");  wait for 1 ns;
181     exp := x"DEADFEED800081356969ABADCAB0AEFC";
182     assert rd = exp report "LI in Lane 1 overwrite mismatched..." severity error;
183     rs1 <= rd;  wait for 1 ns;
184
185     -- (Optional) Spacer/reset section   using '-' may propagate unknowns in sim.
186     -- Prefer zeros to keep waves clean.
```

```
187     instruction <= (others => '-');  wait for 1 ns;
188     rs1 <= (others => '-');          wait for 1 ns;
189
190     -------------------------------------------------------------
191     -- == R4 INSTRUCTIONS ==
192     -- One or two tests per form:
193     --   000/001: INT add (LOW/HIGH 16x16?32 into 32-bit halves with sat)
194     --   010/011: INT sub (LOW/HIGH 16x16?32 into 32-bit halves with sat)
195     --   100/101: LONG add (choose 32 half, 32x32?64 into 64-bit with sat)
196     --   110/111: LONG sub (...)
197     -------------------------------------------------------------
198
199     -- FORM 000 : INT   LOW   ADD (two 32-bit halves per lane)
200     rs1 <= x"00000000800000017FFFFFFEFFFFFFFF";
201     rs2 <= x"FFFFFFFDFFFF8000FFFF7FFFFFFF0002";
202     rs3 <= x"FFFF0008FFFF0001FFFF7FFFFFFF0003";
203     instruction <= mk_r4("000");
204     wait for 5 ns;
205     exp := x"FFFFFFE8800000007FFFFFFF00000005";
206     assert rd = exp report "FORM 000: INT LOW ADD ERROR ... " severity error;
207
208     -- FORM 001 : INT   HIGH   ADD
209     rs1 <= x"00000000800000017FFFFFFEFFFFFFFF";
210     rs2 <= x"FFFDFFFF8000FFFF7FFFFFFF0002FFFF";
211     rs3 <= x"0008FFFF0001FFFF7FFFFFFF0003FFFF";
212     instruction <= mk_r4("001");
213     wait for 5 ns;
214     exp := x"FFFFFFE8800000007FFFFFFF00000005";
215     assert rd = exp report "FORM 001: INT HIGH ADD ERROR ... " severity error;
216
217     -- FORM 010 : INT   LOW   SUB
218     rs1 <= x"00000000800000107FFFFFF0FFFFFFFE";
219     rs2 <= x"00000000000800000007FFF00000003";
220     rs3 <= x"00000000000080000000080000000FFFF";
221     instruction <= mk_r4("010");
222     wait for 5 ns;
223     exp := x"00000000800000007FFFFFFF00000001";
224     assert rd = exp report "FORM 010: INT LOW SUB ERROR ... " severity error;
225
226     -- FORM 011 : INT   HIGH   SUB
227     rs1 <= x"00000000800000107FFFFFF0FFFFFFFE";
228     rs2 <= x"00000000800000007FFF000000030000";
229     rs3 <= x"00000000800000008000000FFFF0000";
230     instruction <= mk_r4("011");
231     wait for 5 ns;
232     exp := x"00000000800000007FFFFFFF00000001";
233     assert rd = exp report "FORM 011: INT HIGH SUB ERROR ... " severity error;
234
235     -- FORM 100 : LONG   LOW   ADD (32x32?64, 64-bit sat)
236     -- Test 1: positive overflow (top lane) and negative underflow (bottom)
237     rs1 <= x"7FFFFFFFFFFFFFF08000000000000010";
238     rs2 <= x"000000007FFFFFFF0000000080000000";
239     rs3 <= x"000000007FFFFFFF000000007FFFFFFF";
240     instruction <= mk_r4("100");
241     wait for 5 ns;
242     exp := x"7FFFFFFFFFFFFFFF8000000000000000";
243     assert rd = exp report "FORM 100: Sat. LONG LOW ADD ERROR ..." severity error;
244
245     -- Test 2: normal computation (clean numbers)
```

```
246     rs1 <= x"000000000000000B0000000000000079";
247     rs2 <= x"00000000FFFFFFFE00000000FFFFFFFD";
248     rs3 <= x"00000000FFFFFFFA0000000000000002";
249     instruction <= mk_r4("100");
250     wait for 5 ns;
251     exp := x"00000000000000170000000000000073";
252     assert rd = exp report "FORM 100: Comp. LONG LOW ADD ERROR ..." severity error;
253
254     -- FORM 101 : LONG   HIGH   ADD (same as 100 but high32 multiplicands)
255     -- Test 1: saturation
256     rs1 <= x"7FFFFFFFFFFFFFF08000000000000010";
257     rs2 <= x"7FFFFFFF000000008000000000000000";
258     rs3 <= x"7FFFFFFF000000007FFFFFFF00000000";
259     instruction <= mk_r4("101");
260     wait for 5 ns;
261     exp := x"7FFFFFFFFFFFFFFF8000000000000000";
262     assert rd = exp report "FORM 101: Sat. LONG HIGH ADD ERROR ..." severity error;
263
264     -- Test 2: computation
265     rs1 <= x"000000000000000B0000000000000079";
266     rs2 <= x"FFFFFFFE00000000FFFFFFFD00000000";
267     rs3 <= x"FFFFFFFA0000000000000000200000000";
268     instruction <= mk_r4("101");
269     wait for 5 ns;
270     exp := x"00000000000000170000000000000073";
271     assert rd = exp report "FORM 101: Comp. LONG HIGH ADD ERROR ..." severity
            error;
272
273     -- FORM 110 : LONG   LOW   SUB
274     -- Test 1: saturation
275     rs1 <= x"7FFFFFFFFFFFFFF08000000000000010";
276     rs2 <= x"000000008000000000000000007FFFFFFF";
277     rs3 <= x"000000007FFFFFFF000000007FFFFFFF";
278     instruction <= mk_r4("110");
279     wait for 5 ns;
280     exp := x"7FFFFFFFFFFFFFFF8000000000000000";
281     assert rd = exp report "FORM 110: Sat. LONG LOW SUB ERROR ..." severity error;
282
283     -- Test 2: computation
284     rs1 <= x"000000000000000DFFFFFFFFFFFFFC17";
285     rs2 <= x"00000000FFFFFFFA00000000FFFFFF9C";
286     rs3 <= x"00000000FFFFFFFE000000000000000A";
287     instruction <= mk_r4("110");
288     wait for 5 ns;
289     exp := x"0000000000000001FFFFFFFFFFFFFFFF";
290     assert rd = exp report "FORM 110: Comp. LONG LOW SUB ERROR ..." severity error;
291
292     -- FORM 111 : LONG   HIGH   SUB
293     -- Test 1: saturation
294     rs1 <= x"7FFFFFFFFFFFFFF08000000000000010";
295     rs2 <= x"80000000000000007FFFFFFF00000000";
296     rs3 <= x"7FFFFFFF000000007FFFFFFF00000000";
297     instruction <= mk_r4("111");
298     wait for 5 ns;
299     exp := x"7FFFFFFFFFFFFFFF8000000000000000";
300     assert rd = exp report "FORM 111: Sat. LONG HIGH SUB ERROR ..." severity error;
301
302     -- Test 2: computation
303     rs1 <= x"000000000000000DFFFFFFFFFFFFFC17";
```

```
304     rs2 <= x"FFFFFFFA00000000FFFFFF9C00000000";
305     rs3 <= x"FFFFFFFE000000000000000A00000000";
306     instruction <= mk_r4("111");
307     wait for 5 ns;
308     exp := x"0000000000000001FFFFFFFFFFFFFFFF";
309     assert rd = exp report "FORM 111: Comp. LONG HIGH SUB ERROR ..." severity
            error;
310
311     ------------------------------------------------------------------
312     -- == R3 INSTRUCTIONS ==
313     -- Opcodes (per your map):
314     --   0000 NOP     | 0001 SHRHI    | 0010 AU      | 0011 CNT1H
315     --   0100 AHS     | 0101 OR       | 0110 BCW     | 0111 MAXWS
316     --   1000 MINWS   | 1001 MLHU     | 1010 MLHCU   | 1011 AND
317     --   1100 CLZW    | 1101 ROTW     | 1110 SFWU    | 1111 SFHS
318     --
319     -- For ops that *use* instruction(14:10) as an immediate/shift (e.g., SHRHI,
            MLHCU),
320     -- we pass the desired 5-bit value via mk_r3's second arg.
321     ------------------------------------------------------------------
322
323     -- 0000 : NOP (no operation)   rd should be unchanged/zero (given zeroed rs*)
324     rs1 <= x"ABCDEFABCDEFABCD80008135DEADFEED";
325     rs2 <= x"10000100000011111110000000111010";
326     rs3 <= x"10000100000011111110000000111010";
327     instruction <= mk_r3("0000", "00000");
328     wait for 5 ns;
329     exp := x"00000000000000000000000000000000";
330     assert rd = exp report "OPCODE 0000: NOP ERROR ..." severity error;
331
332     -- 0001 : SHRHI (per-halfword logical right shift by imm4 = 31 here)
333     rs1 <= x"ABCDEFABCDEFABCD80008135DEADFEED";
334     rs2 <= (others => '0');
335     rs3 <= (others => '0');
336     instruction <= mk_r3("0001", "11111"); -- shift amount = 31 (lower 4 bits used)
337     wait for 5 ns;
338     exp := x"00010001000100010001000100010001";
339     assert rd = exp report "OPCODE 0001: SHRHI ERROR ..." severity error;
340
341     -- 0010 : AU (add word unsigned)   4 independent 32-bit adds (mod 2^32)
342     rs1 <= x"11111111222222223333333344444444";
343     rs2 <= x"0102030405060708090A0B0C0D0E0F10";
344     rs3 <= (others => '0');
345     instruction <= mk_r3("0010", "11111");
346     wait for 5 ns;
347     exp := x"121314152728292A3C3D3E3F51525354";
348     assert rd = exp report "OPCODE 0010: AU ERROR ..." severity error;
349
350     -- 0011 : CNT1H (count ones per 16-bit halfword)
351     rs1 <= x"0000FFFF5555AAAA80007FFF00FFFF00";
352     rs2 <= x"0102030405060708090A0B0C0D0E0F10";
353     rs3 <= (others => '0');
354     instruction <= mk_r3("0011", "11111");
355     wait for 5 ns;
356     exp := x"00000010000800080001000F00080008";
357     assert rd = exp report "OPCODE CNT1H ERROR ..." severity error;
358
359     -- 0100 : AHS (add halfword signed with saturation)
360     rs1 <= x"FFFF12348000C000400080007FFF0003";
```

```
361     rs2 <= x"0002EDCC8000C0004000FFFF00010004";
362     rs3 <= (others => '0');
363     instruction <= mk_r3("0100", "11111");
364     wait for 5 ns;
365     exp := x"00010000800080007FFF80007FFF0007";
366     assert rd = exp report "OPCODE AHS ERROR ..." severity error;
367
368     -- 0101 : OR (bitwise OR)
369     rs1 <= x"0F0F0F0F0F0F0F0F0F0F0F0F0F0F0F0F";
370     rs2 <= x"F0F0F0F0F0F0F0F0F0F0F0F0F0F0F0F0";
371     rs3 <= (others => '0');
372     instruction <= mk_r3("0101", "11111");
373     wait for 5 ns;
374     exp := x"FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF";
375     assert rd = exp report "OPCODE OR ERROR ..." severity error;
376
377     -- 0110 : BCW (broadcast top word W3 into all 4 words)
378     rs1 <= x"DEADBEEF11223344A1B2C3D401020304";
379     rs2 <= (others => '0');
380     rs3 <= (others => '0');
381     instruction <= mk_r3("0110", "11111");
382     wait for 5 ns;
383     exp := x"DEADBEEFDEADBEEFDEADBEEFDEADBEEF";
384     assert rd = exp report "OPCODE BCW ERROR ..." severity error;
385
386     -- 0111 : MAXWS (per-word signed max; ties pick rs1 per your >=)
387     rs1 <= x"AAAAAAAA55555555000000000FFFFFFF";
388     rs2 <= x"AAAAAAAA44444444FFFFFFFF00000000";
389     rs3 <= (others => '0');
390     instruction <= mk_r3("0111", "11111");
391     wait for 5 ns;
392     exp := x"AAAAAAAA555555550000000000000000";
393     assert rd = exp report "OPCODE MAXWS ERROR ..." severity error;
394
395     -- 1000 : MINWS (per-word signed min; ties pick rs1 per your <=)
396     rs1 <= x"7FFFFFFF80000000FFFFFFFF00000001";
397     rs2 <= x"7FFFFFFE80000001FFFFFFFE00000002";
398     rs3 <= (others => '0');
399     instruction <= mk_r3("1000", "11111");
400     wait for 5 ns;
401     exp := x"7FFFFFFE80000000FFFFFFFE00000001";
402     assert rd = exp report "OPCODE MINWS ERROR ..." severity error;
403
404     -- 1001 : MLHU (low16low16 unsigned ? 32 per word)
405     rs1 <= x"AAAAFFFFBBBB8000CCCC0001DDDD1234";
406     rs2 <= x"EEEEFFFFFFFF80000000000099990002";
407     rs3 <= (others => '0');
408     instruction <= mk_r3("1001", "11111");
409     wait for 5 ns;
410     exp := x"FFFE000140000000000000000002468";
411     assert rd = exp report "OPCODE MLHU ERROR ..." severity error;
412
413     -- 1010 : MLHCU (low16imm5 unsigned ? 32 per word; imm5=5)
414     rs1 <= x"A1A1FFFFB2B21234C3C300FFD4D40003";
415     rs2 <= (others => '0');
416     rs3 <= (others => '0');
417     instruction <= mk_r3("1010", "00101");  -- imm5 = 5
418     wait for 5 ns;
419     exp := x"0004FFFB00005B04000004FB0000000F";
```

```
420     assert rd = exp report "OPCODE MLHCU ERROR ..." severity error;
421
422     -- 1011 : AND (bitwise AND)
423     rs1 <= x"A1A1FFFFB2B21234C3C300FFD4D40003";
424     rs2 <= x"FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF";
425     rs3 <= (others => '0');
426     instruction <= mk_r3("1011", "00101");
427     wait for 5 ns;
428     exp := x"A1A1FFFFB2B21234C3C300FFD4D40003";
429     assert rd = exp report "OPCODE AND ERROR ..." severity error;
430
431     -- 1100 : CLZW (count leading zeros per word)
432     rs1 <= x"80000000400000000000000100000000";
433     rs2 <= (others => '0');
434     rs3 <= (others => '0');
435     instruction <= mk_r3("1100", "00101");
436     wait for 5 ns;
437     exp := x"00000000000000010000001F00000020";
438     assert rd = exp report "OPCODE CLZW ERROR ..." severity error;
439
440     -- 1101 : ROTW (rotate-right per word by rs2[4:0] per word)
441     rs1 <= x"0000000180000000DEADBEEF01234567";
442     rs2 <= x"00000010000001F0000002000000010";
443     rs3 <= (others => '0');
444     instruction <= mk_r3("1101", "00101");
445     wait for 5 ns;
446     exp := x"8000000000000001DEADBEEF45670123";
447     assert rd = exp report "OPCODE ROTW ERROR ..." severity error;
448
449     -- 1110 : SFWU (unsigned per-word: rd = rs2 - rs1)
450     rs2 <= x"A5A5A5A55555AAAA123456780F0F0F0F";
451     rs1 <= x"87BAC9B76432654312344DE844105451";
452     rs3 <= x"1A3DEADBFECAFEFABCDEF23456790101";
453     instruction <= mk_r3("1110", "00101");
454     wait for 5 ns;
455     exp := x"1DEADBEEF123456700000890CAFEBABE";
456     assert rd = exp report "OPCODE SFWU ERROR ..." severity error;
457
458     -- 1111 : SFHS (signed per-halfword with saturation: rd = rs2 - rs1)
459     rs2 <= x"00017FFF80007FFF40007FFF80000000";
460     rs1 <= x"FFFF000100017FFF4000FFFFFFFF0001";
461     rs3 <= (others => '0');
462     instruction <= mk_r3("1111", "00101");
463     wait for 5 ns;
464     exp := x"00027FFE8000000000007FFF8001FFFF";
465     assert rd = exp report "OPCODE SFHS ERROR ..." severity error;
466
467     ------------------------------------------------------------------
468     -- End of suite   clear inputs and finish
469     ------------------------------------------------------------------
470     rs2 <= (others => '0');
471     rs1 <= (others => '0');
472     rs3 <= (others => '0');
473     instruction <= (others => '0');
474     wait for 5 ns;
475
476     wait;
477   end process;
478 end architecture;
```

Listing 10: ALU self-checking testbench

3.2 Register File Testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity registers_tb is
end entity;

architecture sim of registers_tb is

  -- DUT ports
  signal instructioni : std_logic_vector(24 downto 0) := (others => '0');
  signal instruction  : std_logic_vector(24 downto 0);  -- DUT output
  signal clk          : std_logic := '0';

  signal wr_en        : std_logic := '0';
  signal wr_addr      : std_logic_vector(4 downto 0) := (others => '0');
  signal wr_data      : std_logic_vector(127 downto 0) := (others => '0');

  signal rs1          : std_logic_vector(127 downto 0);
  signal rs2          : std_logic_vector(127 downto 0);
  signal rs3          : std_logic_vector(127 downto 0);

  constant TCK        : time := 10 ns;

  -- Patterns:
  -- R16: lane1 = 0xABCD, lane0 = 0xEF01, rest zero
  constant R16_VAL : std_logic_vector(127 downto 0) :=
    x"0000000000000000000000ABCD" & x"EF01";

  -- R17: lane1 = 0xFFFF, lane0 = 0x0A0A, rest zero
  constant R17_VAL : std_logic_vector(127 downto 0) :=
    x"00000000000000000000000FFFF" & x"0A0A";

  -- R18: lane0 = 0x8000, rest zero
  constant R18_VAL : std_logic_vector(127 downto 0) :=
    x"00000000000000000000000000000" & x"8000";

  constant ZERO128 : std_logic_vector(127 downto 0) := (others => '0');

begin

  ------------------------------------------------------------------------
  -- Clock generator
  ------------------------------------------------------------------------
  clk_proc : process
  begin
    while true loop
      clk <= '0';
      wait for TCK/2;
      clk <= '1';
      wait for TCK/2;
    end loop;
```

```
53  end process;
54
55  -------------------------------------------------------------------------
56  -- DUT instantiation
57  -------------------------------------------------------------------------
58  uut : entity work.registers
59    port map(
60      instructioni => instructioni,
61      instruction  => instruction,
62      clk          => clk,
63      wr_en        => wr_en,
64      wr_addr      => wr_addr,
65      wr_data      => wr_data,
66      rs1          => rs1,
67      rs2          => rs2,
68      rs3          => rs3
69    );
70
71  -------------------------------------------------------------------------
72  -- Stimulus
73  -------------------------------------------------------------------------
74  stim_proc : process
75  begin
76    -- Init
77    wr_en        <= '0';
78    wr_addr      <= (others => '0');
79    wr_data      <= (others => '0');
80    instructioni <= (others => '0');
81
82    -- Let things settle
83    wait for 2 * TCK;
84
85    -------------------------------------------------------------------------
86    -- 1) Write R16, R17, R18 via the write port
87    -------------------------------------------------------------------------
88
89    -- Write r16 (addr 16 = "10000") with R16_VAL
90    wr_en   <= '1';
91    wr_addr <= "10000";       -- 16
92    wr_data <= R16_VAL;
93    wait until rising_edge(clk);
94
95    -- Write r17 (addr 17 = "10001") with R17_VAL
96    wr_addr <= "10001";       -- 17
97    wr_data <= R17_VAL;
98    wait until rising_edge(clk);
99
100   -- Write r18 (addr 18 = "10010") with R18_VAL
101   wr_addr <= "10010";       -- 18
102   wr_data <= R18_VAL;
103   wait until rising_edge(clk);
104
105   -- Stop writing
106   wr_en   <= '0';
107   wr_addr <= (others => '0');
108   wr_data <= (others => '0');
109
110   -------------------------------------------------------------------------
111   -- 2) Issue an R4 instruction:
```

```
    --    instruction[24:23] = "10"  => R4 format
    --    rs1 = r16 -> bits [9:5]   = 10000
    --    rs2 = r17 -> bits [14:10] = 10001
    --    rs3 = r18 -> bits [19:15] = 10010
    --    other bits don't matter to the register file
    ------------------------------------------------------------------------
    instructioni <= (others => '0');
    instructioni(24 downto 23) <= "10";        -- R4
    instructioni(9  downto 5)  <= "10000";     -- rs1 = r16
    instructioni(14 downto 10) <= "10001";     -- rs2 = r17
    instructioni(19 downto 15) <= "10010";     -- rs3 = r18

    -- Wait for synchronous read (if your RF is sync), then small delay
    wait until rising_edge(clk);
    wait for 1 ns;

    ------------------------------------------------------------------------
    -- 3) Check results
    ------------------------------------------------------------------------
    assert rs1 = R16_VAL
      report "R4 test: rs1 != R16_VAL (r16)" severity error;

    assert rs2 = R17_VAL
      report "R4 test: rs2 != R17_VAL (r17)" severity error;

    assert rs3 = R18_VAL
      report "R4 test: rs3 != R18_VAL (r18)" severity error;

    report "Register file R4 rs1/rs2/rs3 test PASSED." severity note;

    -- Keep sim running so you can inspect waveforms
    wait;
  end process;

end architecture;
```

Listing 11: Register file testbench

3.3 Instruction Buffer Testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity instr_buffer_tb is
end entity;

architecture sim of instr_buffer_tb is

  -- DUT ports
  signal clk       : std_logic := '0';
  signal rst       : std_logic := '0';
  signal instr_out : std_logic_vector(24 downto 0);
  signal pc_out    : unsigned(5 downto 0);

  constant TCK     : time := 10 ns;

```

```
18    -- helper: convert std_logic_vector to "0101..." string
19    function slv_to_str(s : std_logic_vector) return string is
20      variable result : string(1 to s'length);
21      variable idx    : integer := 1;
22    begin
23      for i in s'range loop
24        if s(i) = '1' then
25          result(idx) := '1';
26        else
27          result(idx) := '0';
28        end if;
29        idx := idx + 1;
30      end loop;
31      return result;
32    end function;
33
34  begin
35
36    ------------------------------------------------------------------------
37    -- Clock generation
38    ------------------------------------------------------------------------
39    clk_proc : process
40    begin
41      while true loop
42        clk <= '0';
43        wait for TCK/2;
44        clk <= '1';
45        wait for TCK/2;
46      end loop;
47    end process;
48
49    ------------------------------------------------------------------------
50    -- DUT instantiation
51    ------------------------------------------------------------------------
52    uut : entity work.instr_buffer
53      port map(
54        clk       => clk,
55        rst       => rst,
56        instr_out => instr_out,
57        pc_out    => pc_out
58      );
59
60    ------------------------------------------------------------------------
61    -- Stimulus
62    ------------------------------------------------------------------------
63    stim_proc : process
64      constant NUM_CYCLES : integer := 20;  -- how many instructions to step through
65    begin
66      ------------------------------------------------------------------------
67      -- Apply reset
68      ------------------------------------------------------------------------
69      rst <= '1';
70      wait for 3 * TCK;  -- hold reset a few cycles
71      rst <= '0';
72
73      ------------------------------------------------------------------------
74      -- Step through a number of instructions, printing PC + instr_out
75      ------------------------------------------------------------------------
76      for n in 0 to NUM_CYCLES-1 loop
```

```
77        wait until rising_edge(clk);
78        wait for 1 ns;  -- small delay to let instr_out/pc_out settle
79
80        report "Cycle " & integer'image(n) &
81               "  PC = " & integer'image(to_integer(pc_out)) &
82               "  instr = " & slv_to_str(instr_out);
83      end loop;
84
85      report "instr_buffer_tb finished after " &
86             integer'image(NUM_CYCLES) & " cycles." severity note;
87
88      -- end simulation
89      assert false report "End of simulation" severity failure;
90    end process;
91
92  end architecture;
```

Listing 12: Instruction buffer testbench

3.4 Data Forwarding Unit Testbench

```
1   library ieee;
2   use ieee.std_logic_1164.all;
3   use ieee.numeric_std.all;
4
5   entity data_forwarding_unit_tb is
6   end entity;
7
8   architecture sim of data_forwarding_unit_tb is
9
10    -- DUT inputs
11    signal instr_id        : std_logic_vector(24 downto 0) := (others => '0');
12    signal rs1i, rs2i, rs3i: std_logic_vector(127 downto 0) := (others => '0');
13
14    signal ex_addr         : std_logic_vector(4 downto 0) := (others => '0');
15    signal ex_data         : std_logic_vector(127 downto 0) := (others => '0');
16    signal ex_we           : std_logic := '0';
17
18    signal wb_addr         : std_logic_vector(4 downto 0) := (others => '0');
19    signal wb_data         : std_logic_vector(127 downto 0) := (others => '0');
20    signal wb_we           : std_logic := '0';
21
22    -- DUT outputs
23    signal rs1, rs2, rs3   : std_logic_vector(127 downto 0);
24
25    -- Some distinct patterns
26    constant BASE_RS1  : std_logic_vector(127 downto 0) :=
          x"11111111111111111111111111111111";
27    constant BASE_RS2  : std_logic_vector(127 downto 0) :=
          x"22222222222222222222222222222222";
28    constant BASE_RS3  : std_logic_vector(127 downto 0) :=
          x"33333333333333333333333333333333";
29
30    constant EX_VAL    : std_logic_vector(127 downto 0) :=
          x"EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE";
31    constant WB_VAL    : std_logic_vector(127 downto 0) :=
          x"--------------------------------";
```

```
32
33 begin
34
35   ----------------------------------------------------------------------------
36   -- DUT instantiation
37   ----------------------------------------------------------------------------
38   uut : entity work.data_forwarding_unit
39     port map(
40       instr_id => instr_id,
41       rs1i     => rs1i,
42       rs2i     => rs2i,
43       rs3i     => rs3i,
44       ex_addr  => ex_addr,
45       ex_data  => ex_data,
46       ex_we    => ex_we,
47       wb_addr  => wb_addr,
48       wb_data  => wb_data,
49       wb_we    => wb_we,
50       rs1      => rs1,
51       rs2      => rs2,
52       rs3      => rs3
53     );
54
55   ----------------------------------------------------------------------------
56   -- Stimulus process
57   ----------------------------------------------------------------------------
58   stim_proc : process
59   begin
60     ----------------------------------------------------------------------------
61     -- Common base values
62     ----------------------------------------------------------------------------
63     rs1i <= BASE_RS1;
64     rs2i <= BASE_RS2;
65     rs3i <= BASE_RS3;
66
67     ex_we   <= '0';
68     wb_we   <= '0';
69     ex_addr <= (others => '0');
70     wb_addr <= (others => '0');
71     ex_data <= (others => '0');
72     wb_data <= (others => '0');
73
74     wait for 10 ns;
75
76     ----------------------------------------------------------------------------
77     -- TEST 1: LI, no forwarding
78     -- instr_id(24) = '0' => LI
79     -- rd (also used as "source" in your logic) in bits [4:0] = 16 ("10000")
80     ----------------------------------------------------------------------------
81     instr_id <= (others => '0');
82     instr_id(24)         <= '0';       -- LI
83     instr_id(4 downto 0) <= "10000";   -- rd / rs1 index
84
85     wait for 5 ns;
86
87     assert rs1 = BASE_RS1 report "TEST1: LI no-forward rs1 wrong" severity error;
88     assert rs2 = BASE_RS2 report "TEST1: LI no-forward rs2 wrong" severity error;
89     assert rs3 = BASE_RS3 report "TEST1: LI no-forward rs3 wrong" severity error;
90
```

```
91      ---------------------------------------------------------------------------
92      -- TEST 2: LI, EX forwarding hit (EX wins)
93      -- ex_we=1, ex_addr=rd, so rs1 should be ex_data
94      ---------------------------------------------------------------------------
95      ex_we   <= '1';
96      ex_addr <= "10000";      -- same as instr_id(4 downto 0)
97      ex_data <= EX_VAL;
98
99      wb_we   <= '1';
100     wb_addr <= "10000";      -- same reg, but EX should win
101     wb_data <= WB_VAL;
102
103     wait for 5 ns;
104
105     assert rs1 = EX_VAL
106       report "TEST2: LI EX-forward failed (rs1 != EX_VAL)" severity error;
107
108     assert rs2 = BASE_RS2
109       report "TEST2: LI EX-forward rs2 changed unexpectedly" severity error;
110
111     assert rs3 = BASE_RS3
112       report "TEST2: LI EX-forward rs3 changed unexpectedly" severity error;
113
114     ---------------------------------------------------------------------------
115     -- TEST 3: LI, no EX but WB forwarding hit
116     ---------------------------------------------------------------------------
117     ex_we <= '0';            -- disable EX
118     wb_we <= '1';            -- WB active
119     wb_addr <= "10000";
120     wb_data <= WB_VAL;
121
122     wait for 5 ns;
123
124     assert rs1 = WB_VAL
125       report "TEST3: LI WB-forward failed (rs1 != WB_VAL)" severity error;
126
127     ---------------------------------------------------------------------------
128     -- TEST 4: R3, EX forwarding on rs1 and rs2
129     -- instr_id(24:23) = "11"
130     -- rs1 index = bits [9:5] = 17 ("10001")
131     -- rs2 index = bits [14:10] = 18 ("10010")
132     ---------------------------------------------------------------------------
133     rs1i <= BASE_RS1;
134     rs2i <= BASE_RS2;
135     rs3i <= BASE_RS3;
136
137     instr_id <= (others => '0');
138     instr_id(24 downto 23) <= "11";      -- R3
139     instr_id(9  downto 5)  <= "10001";   -- rs1 idx = 17
140     instr_id(14 downto 10) <= "10010";   -- rs2 idx = 18
141
142     -- EX forwarding to rs1 only
143     ex_we   <= '1';
144     ex_addr <= "10001";      -- matches rs1 index
145     ex_data <= EX_VAL;
146
147     -- WB forwarding to rs2 only
148     wb_we   <= '1';
149     wb_addr <= "10010";      -- matches rs2 index
```

```
150     wb_data <= WB_VAL;
151
152     wait for 5 ns;
153
154     assert rs1 = EX_VAL
155       report "TEST4: R3 EX-forward to rs1 failed" severity error;
156     assert rs2 = WB_VAL
157       report "TEST4: R3 WB-forward to rs2 failed" severity error;
158     assert rs3 = BASE_RS3
159       report "TEST4: R3 unexpected change to rs3" severity error;
160
161     ----------------------------------------------------------------------
162     -- TEST 5: R3, EX vs WB conflict on rs1 (EX priority)
163     ----------------------------------------------------------------------
164     rs1i <= BASE_RS1;
165     rs2i <= BASE_RS2;
166     rs3i <= BASE_RS3;
167
168     instr_id <= (others => '0');
169     instr_id(24 downto 23) <= "11";      -- R3
170     instr_id(9  downto 5)  <= "01010";   -- rs1 index = 10
171
172     ex_we   <= '1';
173     ex_addr <= "01010";
174     ex_data <= EX_VAL;
175
176     wb_we   <= '1';
177     wb_addr <= "01010";
178     wb_data <= WB_VAL;
179
180     wait for 5 ns;
181
182     assert rs1 = EX_VAL
183       report "TEST5: R3 EX vs WB conflict: EX did not win for rs1" severity error;
184
185     ----------------------------------------------------------------------
186     -- All tests done
187     ----------------------------------------------------------------------
188     report "All data_forwarding_unit tests PASSED." severity note;
189     wait;
190   end process;
191
192 end architecture;
```

Listing 13: Data forwarding unit testbench

3.5 Top-Level Pipeline Testbench

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity tb_multimedia_pipeline is
6  end entity;
7
8  architecture sim of tb_multimedia_pipeline is
9
```

```
10    signal clk_if, clk_id, clk_ex, clk_wb : std_logic := '0';
11    signal rst       : std_logic := '0';
12
13    signal rd_out       : std_logic_vector(127 downto 0);
14    signal instr_id_out : std_logic_vector(24 downto 0);
15    signal pc_if_out    : unsigned(5 downto 0);
16    signal wr_en_dbg    : std_logic;
17    signal df_addr_dbg  : std_logic_vector(4 downto 0);
18
19    constant PHASE : time := 5 ns;  -- spacing between edges
20
21  begin
22
23    ---------------------------------------------------------------------
24    -- DUT
25    ---------------------------------------------------------------------
26    U_DUT : entity work.pipelined_simd_multimedia_unit
27      port map(
28        clk_if       => clk_if,
29        clk_id       => clk_id,
30        clk_ex       => clk_ex,
31        clk_wb       => clk_wb,
32        rst          => rst,
33        rd_out       => rd_out,
34        instr_id_out => instr_id_out,
35        pc_if_out    => pc_if_out,
36        wr_en_dbg    => wr_en_dbg,
37        df_addr_dbg  => df_addr_dbg
38      );
39
40    ---------------------------------------------------------------------
41    -- 4-phase clock generator: IF ? ID ? EX ? WB
42    ---------------------------------------------------------------------
43    clk_proc : process
44    begin
45      -- apply reset
46      rst <= '1';
47      clk_if <= '0'; clk_id <= '0'; clk_ex <= '0'; clk_wb <= '0';
48      wait for 4*PHASE;
49      rst <= '0';
50
51      -- run for some steps
52      for step in 0 to 40 loop
53        -- IF
54        clk_if <= '1'; wait for PHASE; clk_if <= '0'; wait for PHASE;
55        -- ID
56        clk_id <= '1'; wait for PHASE; clk_id <= '0'; wait for PHASE;
57        -- EX
58        clk_ex <= '1'; wait for PHASE; clk_ex <= '0'; wait for PHASE;
59        -- WB
60        clk_wb <= '1'; wait for PHASE; clk_wb <= '0'; wait for PHASE;
61      end loop;
62
63      assert false report "tb_multimedia_pipeline finished"
64        severity failure;
65    end process;
66
67    ---------------------------------------------------------------------
68    -- Monitor WB events (every WB rising edge)
```

```
69    -----------------------------------------------------------------
70    monitor_proc : process(clk_wb)
71      variable low16 : unsigned(15 downto 0);
72    begin
73      if rising_edge(clk_wb) then
74        low16 := unsigned(rd_out(15 downto 0));
75        report "WB: PC_IF=" & integer'image(to_integer(pc_if_out)) &
76               " instr_id=" & to_hstring(instr_id_out) &
77               " wr_en=" & std_logic'image(wr_en_dbg) &
78               " wb_addr=" & integer'image(to_integer(unsigned(df_addr_dbg))) &
79               " rd_out[15:0]=" & integer'image(to_integer(low16));
80      end if;
81    end process;
82
83  end architecture;
```

Listing 14: Top-level pipelined SIMD multimedia unit testbench